

ZYVORAI LABS · CUSTOMER DOCUMENTATION

Zyvor Fabric

Page-by-Page Product Manual

2026-09-01

Page-by-page guides

Each guide follows: Purpose → When to use it → How to get there → What you can do → Related pages.

Every route is also listed in the [complete page index](#).

Auth

Page	What it covers
Sign in	Sign-in screen for Zylvor Fabric. Authenticates against the local admin account or a configured identity provider / PAM user on the host.

Core

Page	What it covers
Create VM	Create VM — a three-step wizard (Basics → Resources → Review) for launching a new virtual machine, including how it's networked and whether it's reachable from outside the host.
Datacenters	Datacenters — the physical inventory tree: datacenters, the clusters inside each one, and the hosts registered to each cluster, with live CPU/memory usage and VM counts per host.
Favorites	Favorites — a personal, starred shortlist of VMs pulled from your full VM list, so the machines you use most are one click away instead of buried in a longer list.
Dashboard	Dashboard — the fabric at a glance: how many VMs exist and in what state, live CPU/memory trends, and whether each backend subsystem is actually reachable.
Machines	Machines — a lower-level view of the VM driver's running instances and the raw disk images it can boot, with direct shell access into a running instance.
Profiles	Profiles (shown in the UI as Instance Types) — a library of VM sizing presets (vCPUs, memory, disk, and optionally network bandwidth) you can pick instead of hand-tuning resources every time you create a VM.
Settings	Settings — Core product and console preferences.
VM Browser	VM Browser — a lightweight, read-only grid of every VM, for quickly scanning or searching without the bulk-action tooling of the full Virtual Machines list.
VM Wizard	VM Wizard — guided VM creation flow.
VM Console	VM Console — a real, live console into a running VM, from the browser, without SSH or any other client installed.
Virtual Machines	Virtual Machines — the fleet view of every VM in the fabric, and the starting point for managing any one of them.

Infrastructure

Page	What it covers
Containers	Containers — a read-only, auto-refreshing view of container workloads running on the host (Docker/Podman-style containers, distinct from VMs), showing per-container state, image, CPU/memory usage, and network I/O.

Page	What it covers
Distributed Storage	Distributed Storage — the enterprise/clustered layer above a single storage backend: replicated pools spanning multiple hosts, storage policies (tiering, replication, encryption/dedup/compression), in-flight VM disk migrations between pools, and datastore clusters with automatic space/latency-based balancing. For creating and starting a single NFS/LVM/ZFS/Ceph pool, see Storage Pools ; for browsing volumes inside pools, see Storage .
Net Security	Net Security — the advanced SDN and security control plane: network policies scoped to security identities, host firewall profiles/zones/VM assignments, exposed services, QoS traffic shaping, DNS zones/policies, WireGuard VPN tunnels and networks, traffic mirroring, NAT rules/pools/gateways, and bandwidth monitoring with alerts. For everyday per-VM networking mode and port forwards, see Network instead.
Network	Network — day-to-day VM networking: which mode a VM uses, its port forwards, and its assigned address. For the advanced SDN stack (policies, firewalls, VPN mesh, QoS, mirroring), see Net Security instead.
Resource Pools	Resource Pools — hierarchical CPU/memory allocation pools (nested, with shares, reservations, and limits) that VMs draw from, plus an admission-control test to check whether a workload's requirements would fit before you commit to it.
Storage Pools	Storage Pools — create, start/stop, and monitor the storage backends VM disks live on: local directories, NFS exports, LVM and LVM-thin volume groups, ZFS pools, or Ceph RBD pools. This is where a pool's lifecycle and health live; for the volumes (disks) inside those pools see Storage , and for multi-host replicated/policy-driven storage see Distributed Storage .
Storage	Storage — a consolidated view of every storage pool's capacity alongside a manual volume tracking ledger. To create or manage a pool itself, use Storage Pools ; for replicated/policy-driven storage across hosts, see Distributed Storage .
System Health	System Health — a live, read-only dashboard of host resource utilization, refreshing every 2 seconds: CPU, memory, disk I/O, filesystems, network interfaces, and top processes, rolled up into a single health score.
System	System — the physical host's hardware topology (CPU sockets/cores/threads, NUMA nodes, hugepages) plus topology-aware optimization recommendations for individual VMs. This is distinct from System Health , which tracks live utilization rather than hardware layout.
VM Pools	VM Pools — warm / capacity pools for fast provisioning.
Zones	Zones — Infrastructure surface for placement domains.

Monitoring

Page	What it covers
Alerts	Alerts — a live view of currently firing system alerts and the notification rules that generate them, polling for updates automatically.
Analytics	Performance Analytics — fleet-wide resource utilization, trends over time, and per-VM performance insights, with exportable reports.
Audit	Audit Logs — the security and compliance trail of who did what: every tracked action, which user performed it, on which resource, whether it succeeded, and from what IP address.
Capacity	Capacity Planning — resource usage against total capacity per resource (memory, CPU, storage), with week-over-week trend, so you can see what's running out and when.

Page	What it covers
Debug Tools	Debug Tools — raw, terminal-style output from four classic Linux diagnostic commands (top, iostat, vmstat, netstat) against the host, rendered as monospace panels. This is the closest the dashboard gets to SSHing in and running commands yourself.
Event Stream	Event Stream — a live, scrolling log of VM lifecycle events (create, start, stop, delete, and similar) pushed over an authenticated SSE connection as they happen. There's no history — you only see events that occur while the page is open.
Explain	Explain — plain-language, AI-generated explanations for a chosen system metric: its current value and trend, an assessment of its status, what's contributing to it, and what to do about it. This is the interpretive layer on top of the raw numbers you'd see in Analytics or Debug Tools.
Kernel	Kernel — a snapshot of the host's kernel configuration: version, hostname, architecture, boot command line, loaded kernel modules, and sysctl parameters. This is static configuration, not live activity — for that, see Debug Tools or Event Stream.
Live Metrics	Live Metrics — a real-time view of host performance: CPU, memory, disk I/O, and network throughput, each as a rolling sparkline that updates once per second.
Logs	Logs — a searchable, filterable console view of Zyvor Fabric's audit log: every recorded action and event, level-coded and continuously refreshed.
Notifications	Notifications — configure how and when Zyvor Fabric alerts you: the delivery channels (email, Slack, Teams, webhook), the rules that trigger them, and a history of what was actually sent.
Processes	Processes — a live process monitor for the host: every OS process with its CPU and memory usage, refreshed every 3 seconds, with a per-process detail drill-down.
Optimizer	Optimizer — a right-sizing advisor that analyzes each VM's actual resource usage and recommends CPU/memory/disk adjustments, with a one-click apply per VM.
Service Map	Service Map — shows the services discovered across your VMs, which ones depend on each other (protocol and port), and each service's current health.
Timeline	Timeline — a single reverse-chronological activity feed that merges audit-log actions and system alerts, so you can see what happened and in what order without switching between Logs and Notifications.

More Images Migrations Managers

Page	What it covers
Backup Scheduler	Backup Scheduler — create and manage recurring, automated backup jobs that snapshot one or more VMs' disks to a directory on a schedule, with retention and format controls.
Batch Import	Batch Import — bulk-create VMs from a YAML or JSON list, with a preview step and a per-VM status readout as each one is submitted.
Batch Migration	Batch Migration Builder — a form-based editor for assembling a multi-VM migration job spec (source disk, target format, sizing) and exporting it as JSON. It builds the spec only; it does not run the migration itself.
Disk Converter	Disk Format Converter — convert a single disk image between QCOW2, VMDK, VHD, VHDX, and RAW, tracking the conversion job's progress to completion.
Disk Images	Disk Images — a read-only inventory of the VM disk images present on the host: name, format, size, and path for each one.

Page	What it covers
Download Disk	Download Disk — browse the disk images available on the Fabric host and download any of them straight to your machine.
Image Builder	Image Builder — build custom VM disk images from scratch using mkosi , by picking a Linux distribution and a package list, and track builds from queued through to a finished image.
ISO Images	ISO Images — a read-only inventory of installer and driver ISO files sitting in the host's configured images directory, showing which VMs currently have each one attached.
Job Monitor	Job Monitor — a live view of background jobs (disk conversions, migrations, and other pipeline work), with per-job progress, pipeline stage, and streaming logs.
Manifest Builder	Manifest Builder — a client-side form for assembling a VM configuration manifest and exporting it as YAML, with a live preview as you type. It doesn't create a VM or call the API; it's a scratchpad for drafting config to copy elsewhere.
History	Migration History — a read-only log of completed and failed migration jobs, with status, timing, and where the output landed.
Readiness	Migration Readiness — pre-flight checks that verify the environment is in a good state before you start a migration, with a pass/fail summary and per-check detail.
Report	Migration Report — a shareable summary of all migration jobs (totals by status, average duration) plus the full per-migration detail table, with copy and print actions.
Migration Templates	Migration Templates — reusable migration configuration presets (disk format, vCPUs, memory, network, compression) that you can copy as JSON instead of re-entering the same settings for every migration.
Network Topology	Network Topology — a live map of which VMs are attached to which virtual networks or host bridges, alongside the host's own bridges and physical NICs, auto-refreshing every 15 seconds.
Pipeline	Pipeline Monitor — a live, auto-refreshing view of in-progress migration/conversion jobs, showing each job's percent complete and which of five stages it's currently in.
Snapshot Mgr	Snapshot Manager — create, revert to, and delete disk-state snapshots for a selected VM.
Storage Mgr	Storage Manager — browse storage pools with their capacity and usage, and drill into a pool to see its volumes.
Upload Disk	Upload Disk Image — drag-and-drop (or browse) upload of a VM disk image file to the server, with live progress and an in-session upload history.

Operations

Page	What it covers
Autoscale	Autoscale — define per-VM policies that automatically grow or shrink a VM's vCPUs and memory within set bounds based on load, and review the history of scaling actions that were triggered.
Backups	Backups & Restore — create full or incremental VM backups, track running backup/restore jobs, and restore a VM from a completed backup, either in place or as a new VM.
Bulk Operations	Bulk Operations — select any number of VMs and start, stop, restart, or snapshot them together, with a per-VM progress log for the batch.
Content Library	Content Library — a catalog of reusable provisioning building blocks: libraries of templates/ISOs/OVFs/scripts, guest customization specs (per-OS hostname/domain/DNS settings), and host compliance profiles.

Page	What it covers
DRS	Distributed Resource Scheduler (DRS) — balances VM placement across the hosts in a cluster, surfaces migration recommendations, enforces affinity/anti-affinity rules, and can test where a new VM would land before you create it.
Fault Tolerance	Fault Tolerance (FT) — protect individual VMs with a live secondary replica on another host, so a host failure fails the VM over instead of taking it down, and monitor replication health.
Lifecycle	Lifecycle Manager — define patch/upgrade baselines, scan hosts for compliance against them, remediate non-compliant hosts, and track rolling updates across a host fleet.
Migration Wizard	Migration Wizard — a three-step wizard (Source → Configure → Review) for converting an existing disk image (local file or remote host) into a new Zvvor Fabric VM.
Migrations	VM Migrations — move a VM from its current host to a different target host, and track the migration from start to finish. The list auto-refreshes every 5 seconds so in-flight migrations update live.
Quotas	Resource Quotas — cap CPU, memory, disk, and VM-count usage, applied either globally or to VMs matching specific tags, so a team or workload can't consume unlimited host resources.
Replication	Replication — register remote replication sites, configure per-VM replication to them with a target RPO (recovery point objective), and monitor sync health and RPO compliance across your fleet.
Schedules	VM Schedules — automate a recurring lifecycle action (start, stop, restart, or snapshot) for a single VM, on a one-time, daily, or weekly schedule.
Site Recovery	Site Recovery — define disaster recovery plans that group VMs by source and target site, then execute those plans as a test failover, a planned migration, or a full disaster recovery, and track how each execution unfolds.
Snapshots	VM Snapshots — create point-in-time snapshots of a specific VM's disk (or disk + state), and revert or delete them. Unlike most Operations pages, this one is scoped to one VM at a time, entered by name.
Templates	VM Templates — reusable VM configurations (CPU/memory/disk, tags) that you save from an existing VM and use to stamp out new VMs quickly, instead of configuring resources from scratch each time.

Security

Page	What it covers
Access Control	Access Control — manage the user accounts that can sign in to Zvvor Fabric: create accounts, assign a role (admin, operator, or viewer), and enable/disable or delete them.
Certificates	Certificates & Security — a PKI console for Zvvor Fabric: certificate authorities, issued certificates, the CSR approval queue, host TPM/boot attestation, and VM security baselines, rolled up into one health dashboard.
Compliance	Compliance Dashboard — a security and configuration compliance scorecard: an overall score, pass/warning/fail counts by category, and remediation guidance for anything that isn't passing.
Encryption	Encryption — manage VM disk and vMotion encryption: register key management providers (KMIP, HashiCorp Vault Transit, or local software keys), define reusable encryption policies, and see which VMs are encrypted under which policy.
Plugins	Plugin Manager — enable, disable, and review the server extensions installed on Zvvor Fabric (storage, network, security, monitoring, and backup plugin types).

Page	What it covers
Security Dashboard	Security — a real-time security posture and threat-monitoring view: an overall risk score, active security alerts, recent failed login attempts, and listening network ports on the host. Data auto-refreshes every 5 seconds. This is distinct from Compliance (configuration checks) and Certificates (PKI health) — this page is about live threats and activity, not point-in-time audits.

Tools

Page	What it covers
Cost Estimator	Storage Cost Estimator — a what-if calculator that projects cloud storage cost (AWS S3, Azure Blob, GCS) for your VM fleet and compares it against an on-premises baseline. It's a planning tool: figures come from the inputs you set, not from your actual billing or usage.
Notification Center	Notification Center — a live, session-only tray of VM events and system alerts, polled from the server every 10 seconds. It's separate from Monitoring → Notifications, which manages persistent delivery channels, rules, and history; this page just surfaces what's firing right now and forgets everything when you reload.
API Playground	API Playground — try Fabric APIs interactively.
VM Compare	VM Comparison — a side-by-side diff of two VMs' configurations, run on demand against the live VM list.
VM Health Check	VM Health Check — runs a set of health verification checks against a single VM, on demand, and reports pass/warning/fail per check plus an overall status.
Webhooks	Webhook Configuration — manage outbound webhooks that notify an external endpoint (generic HTTP, Slack, or Discord) when specific VM and backup events occur.

84 guides. Regenerate: `node scripts/customer-docs/generate-guide-index.mjs` .

Sign in

Purpose

Sign-in screen for Zyvor Fabric. Authenticates against the local admin account or a configured identity provider / PAM user on the host.

When to use it

- Whenever you're not signed in — protected `/app` routes redirect here if the session is missing or expired
- To sign in as the local admin, or with your own system account

How to get there

- Route: `/sign-in` (legacy `/login` redirects here)
- From marketing pages: **Sign in** in the top nav
- After signing in you land on the console at `/app`

Operate from the console (UX)

1. Enter a username and password and sign in — a failed attempt shows an inline error.
2. On success you are taken to `/app` (dashboard).
3. Two common ways to sign in: **local admin** — username `admin`, password from `.admin_password` on the host — or your own **system user** account when PAM/OIDC is configured.

If you can't sign in, confirm the account exists and that `zyvor-fabricd` is reachable.

Related pages

- Marketing home: `/`
 - Console dashboard: `/app`
 - [Getting Started](#)
 - [Page index](#)
-

Create VM

Purpose

Create VM — a three-step wizard (Basics → Resources → Review) for launching a new virtual machine, including how it's networked and whether it's reachable from outside the host.

When to use it

- To provision a new VM from a catalog image or a custom disk path
- To decide up front how the VM will be reachable — private-and-forwarded, or a real address on your network
- Start from the product home / dashboard if you are unsure where to begin

How to get there

- Route / id: `/app/create`
- Nav: **Core** → **Create VM** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Basics — name the VM and pick a disk image, either from the built-in catalog or a path on the host.

2. Resources — set vCPUs, memory, and root disk size. Under **Advanced Options** you choose networking:

- **NAT (default)** — the VM gets a private, outbound-only address, same as most VMs today. To make a service on the VM reachable from your laptop or the wider network, use **Expose ports**: add a host port → guest port mapping (there's a one-click **Expose SSH (22)** preset for the common case of just needing to SSH in). Forwarded ports are reachable from any client that can reach the host — not just from the host itself.
- **Bridged** — the VM gets its own real address on the local network via a dedicated network namespace, instead of sharing the host's NAT. Two addressing options:
 - **DHCP** (default under Bridged) — the VM's guest agent leases an address automatically.
 - **Assign the IP statically via cloud-init** — check this box to bake a fixed address into the VM at boot instead of waiting on DHCP, useful when the guest image doesn't run a DHCP client or you need a predictable address before first boot.

3. Review — confirms name, image, resources, and the networking mode you chose, then creates the VM.

After creation — port forwards aren't locked in at creation time: open the VM's **Network** tab from its detail page to add or remove forwards later (the VM restarts automatically if it's running when you do).

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Virtual Machines](#)
 - [Network](#)
 - [Getting Started](#)
 - [Page index](#)
-

Datacenters

Purpose

Datacenters — the physical inventory tree: datacenters, the clusters inside each one, and the hosts registered to each cluster, with live CPU/memory usage and VM counts per host.

When to use it

- To see how your infrastructure is organized — datacenter → cluster → host — before deciding where new capacity should go
- To register a new host into a cluster, or stand up a new datacenter or cluster
- To check a host's CPU/memory load, VM count, or connection status
- To put a host into maintenance mode before doing physical work on it

How to get there

- Route / id: `/app/datacenters`
- Nav: **Core** → **Datacenters** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Read the top-of-page summary cards: total datacenters, clusters, hosts, and VMs across the whole fabric.
2. Click a datacenter to expand it — you'll see a rollup (cluster count, host count, VM count, and, once summary data loads, total CPUs and memory) plus its list of clusters.
3. **Create a datacenter** with the button in the header — a modal asks for a name and optional description.
4. Inside an expanded datacenter, click **+ Cluster** to create a cluster there (name + description), or the trash icon to delete the datacenter — both are confirmed first.
5. Expand a cluster to see its HA and DRS status and a table of its hosts: hostname, address, CPUs, memory, live CPU/memory usage bars, VM count, and status (Connected, Disconnected, Maintenance).
6. Click **+ Host** on a cluster to register a host (hostname, IP address, CPUs, memory in MB), or use the trash icon to remove one — removal is confirmed first.
7. Toggle a host in or out of maintenance mode with the wrench icon next to it.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Favorites

Purpose

Favorites — a personal, starred shortlist of VMs pulled from your full VM list, so the machines you use most are one click away instead of buried in a longer list.

When to use it

- To jump straight to the handful of VMs you check on or console into regularly
- To pin or unpin VMs as your day-to-day working set changes
- To search across every VM (not just the pinned ones) from a single search box

How to get there

- Route / id: `/app/favorites`
- Nav: **Core** → **Favorites** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Click the star next to any VM to pin or unpin it. Favorites are saved in your browser's local storage, not on the server — they don't follow you to a different browser or device.
2. The list splits into **Pinned VMs** (your favorites) and **All VMs** (everything else); the search box filters both by name or state.
3. Each row shows vCPU count, memory, and a state badge; click the name to open the VM's detail page, or **Console** to jump straight into its console tab.
4. **Refresh** reloads the underlying VM list from the server.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Dashboard

Purpose

Dashboard — the fabric at a glance: how many VMs exist and in what state, live CPU/memory trends, and whether each backend subsystem is actually reachable.

When to use it

- As your console landing page — it loads at `/app`
- To check overall health before digging into a specific VM or subsystem
- Start here if you're new to the product; a first-time install lands somewhere useful, not an empty table

How to get there

- Route: `/app`
- Nav: **Core** → **Dashboard** (sidebar or command palette), or the wordmark in the top bar
- After sign-in at `/sign-in` you land here

Operate from the console (UX)

1. Check subsystem status (VM driver, storage, network security, authentication, events) — each shows Live, Unreachable, or Off.
2. Read the stat cards: total VMs, running, stopped, and total allocated memory/vCPUs.
3. Watch live CPU and memory usage charts when VMs are running.
4. Scan the VM table, or open [Virtual Machines](#).
5. **On a fresh install with no VMs yet**, use Getting Started links to create a VM, templates, API playground, or access control.

Related pages

- [Virtual Machines](#)
 - [Create VM](#)
 - [Getting Started](#)
 - [Page index](#)
-

Machines

Purpose

Machines — a lower-level view of the VM driver's running instances and the raw disk images it can boot, with direct shell access into a running instance.

When to use it

- To inspect a running instance's properties (state, leader, class, service, VSocket CID) or grab its ready-made SSH command
- To run a one-off shell command against an instance without opening a full console
- To reboot, power off, or forcibly terminate an instance
- To pull a new raw image from a URL, or remove one you no longer need

How to get there

- Route / id: `/app/machines`
- Nav: **Core** → **Machines** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

The page has two tabs, and refreshes itself every 10 seconds:

Running Machines

1. Click a machine in the left list to load its details on the right — properties (State, Leader, Class, Service, VSocketCID) and, if available, a ready-to-copy SSH command.
2. Use the header buttons to **Reboot**, **Poweroff**, or **Kill** (force terminate) the selected machine.
3. Use the **Shell** panel to run a single command against the selected machine — type it and hit Enter or **Run**; output shows stdout, stderr, and the exit code.

Images

1. **Pull Image** — provide a source URL and a name to pull a new raw disk image.
2. The image table lists name, type, size, and whether it's read-only; remove one with the trash icon.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Profiles

Purpose

Profiles (shown in the UI as **Instance Types**) — a library of VM sizing presets (vCPUs, memory, disk, and optionally network bandwidth) you can pick instead of hand-tuning resources every time you create a VM.

When to use it

- To review available instance-type presets and their specs, grouped by category (general, compute, memory, storage, GPU)
- To create a reusable custom profile for a sizing you use often
- To remove a custom profile you no longer need — built-in profiles can't be deleted

How to get there

- Route / id: `/app/profiles`
- Nav: **Core** → **Profiles** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Browse profile cards — each shows its category badge, description, CPUs, memory, disk, and network bandwidth (if set). Built-in profiles are labeled **Built-in** and have no delete button.
2. **Create Profile** opens a form for a name, category, CPU count, memory (MB), and disk (GB); submitting adds it to the grid immediately.
3. Delete a custom profile with the trash icon on its card — built-in profiles can't be removed this way.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Settings

Purpose

Settings — Core product and console preferences.

When to use it

- Open this surface when the job matches the purpose above
- Start from the product home / dashboard if you are unsure where to begin
- Confirm auth and that required backends/operators are reachable if data looks empty

How to get there

- Route / id: `/app/settings`
- Nav: **Core** → **Settings** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Open `/app/settings` and wait for live data from Zyvor Fabric.
2. Use filters and search when the page provides them.
3. Drill into a row or card for detail, then jump to related surfaces.
4. For mutating actions: review impact, role gates, and confirmation dialogs first.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

VM Browser

Purpose

VM Browser — a lightweight, read-only grid of every VM, for quickly scanning or searching without the bulk-action tooling of the full [Virtual Machines](#) list.

When to use it

- To browse or search VMs by name, state, or image in a compact card layout
- To jump straight to a single VM's detail page without needing selection, filtering by tag, or bulk actions

How to get there

- Route / id: `/app/vm-browser`
- Nav: **Core** → **VM Browser** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Search by name, state, or image — the header shows total VM count and how many are running.
2. Each card shows the VM's name, state badge, image, vCPU count, memory, and IP address (if assigned).
3. Click a card to open that VM's detail page.
4. **Refresh** reloads the list from the server.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

VM Wizard

Purpose

VM Wizard — guided VM creation flow.

When to use it

- Open this surface when the job matches the purpose above
- Start from the product home / dashboard if you are unsure where to begin
- Confirm auth and that required backends/operators are reachable if data looks empty

How to get there

- Route / id: `/vm-wizard`
- Nav: **Core** → **VM Wizard** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Open `/vm-wizard` and wait for live data from Zyvor Fabric.
2. Use filters and search when the page provides them.
3. Drill into a row or card for detail, then jump to related surfaces.
4. For mutating actions: review impact, role gates, and confirmation dialogs first.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

VM Console

Purpose

VM Console — a real, live console into a running VM, from the browser, without SSH or any other client installed.

When to use it

- To reach a VM that isn't network-reachable yet (no IP assigned, no SSH configured)
- To watch or interact with boot output, a serial console, or a full graphical desktop
- To debug a VM that's otherwise unresponsive over the network

How to get there

- Route / id: `/app/vms/:name/console`
- From a VM's detail page, click **Console** in the header
- Nav: reached via **Core → Virtual Machines**, not linked directly from the top nav

Operate from the console (UX)

The page has two tabs:

1. **Terminal** — a real interactive shell (xterm.js) streamed over the VM's PTY. Type as you would in any terminal; output renders live.
2. **VNC** — a real graphical framebuffer session (noVNC), for VMs where a text console isn't enough — installers, desktop environments, or anything that draws to the screen. Shows actual rendered pixels, not just a placeholder.

Both are authenticated with your existing session — no separate console password or exposed port.

If the console tab stays black or disconnected, confirm the VM is running and that the host's console/VNC services are reachable — check service health if it persists.

5. **Empty / fail:** Check health, auth, and domain dependencies.
6. **Success:** Live data loads; mutations complete without error toasts.

Related pages

- [Virtual Machines](#)
 - [Getting Started](#)
 - [Page index](#)
-

Virtual Machines

Purpose

Virtual Machines — the fleet view of every VM in the fabric, and the starting point for managing any one of them.

When to use it

- To see everything running (and everything stopped) at a glance
- To find a specific VM quickly once you have more than a handful
- To act on several VMs at once (start, stop, delete, archive)
- Start from the product home / dashboard if you are unsure where to begin

How to get there

- Route / id: `/app/vms`
- Nav: **Core** → **Virtual Machines** (sidebar, command palette, or desktop nav)
- A single VM's detail page is `/app/vms/:name`, reached by clicking its name here

Operate from the console (UX)

On the list

1. Switch between grid and table view with the toggle in the top right.
2. Search by name, image, or tag — the search box also shows "N of M VMs" so you always know if you're looking at a filtered subset.
3. Filter by tag using the tag chips; combine with search.
4. Select one or more VMs with the checkboxes, then use the bulk actions bar to start, stop, delete, or archive them together.
5. Copy a VM's name straight from the list — hover the name and click the copy icon that appears next to it.
6. Click a VM's name to open its detail page.

On a VM's detail page (`/app/vms/:name`)

1. Start, stop, pause, resume, restart, or clone the VM from the header actions.
2. Open a real console — either the browser terminal (xterm.js) or a graphical VNC session — from the Console tab.
3. Manage disks, network (including port forwards), snapshots, hotplug devices, and cloud-init from their respective tabs.
4. **Deleting a VM is undoable for a few seconds.** Confirming delete doesn't remove the VM immediately — a bar appears at the bottom of the screen with an **Undo** button and a countdown; the VM is only actually deleted once that countdown finishes unclicked. If you change your mind, click Undo before it runs out.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Create VM](#)
- [VM Console](#)
- [Getting Started](#)

- [Page index](#)
-

Containers

Purpose

Containers — a read-only, auto-refreshing view of container workloads running on the host (Docker/Podman-style containers, distinct from VMs), showing per-container state, image, CPU/memory usage, and network I/O.

When to use it

- To check whether container workloads on this host are running, restarting, or exited
- To spot a container consuming excessive CPU or memory before it starves VMs sharing the same host
- To confirm a container's network throughput (RX/TX)

How to get there

- Route / id: `/app/containers`
- Nav: **Infrastructure** → **Containers** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review the summary cards: total containers, how many are running, total CPU%, and total memory across every container detected.
2. Scan the container grid — each card shows the container's name/ID, a state badge (running / exited / paused / restarting), its image, live CPU and memory usage bars, and RX/TX network counters when available.
3. Data refreshes automatically every 3 seconds; use the header refresh button to force an immediate reload.

This page is monitoring-only — there's no start/stop/restart or delete action for containers here.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Distributed Storage

Purpose

Distributed Storage — the enterprise/clustered layer above a single storage backend: replicated pools spanning multiple hosts, storage policies (tiering, replication, encryption/dedup/compression), in-flight VM disk migrations between pools, and datastore clusters with automatic space/latency-based balancing. For creating and starting a single NFS/LVM/ZFS/Ceph pool, see [Storage Pools](#); for browsing volumes inside pools, see [Storage](#).

When to use it

- To provision a storage pool replicated across multiple hosts with a target replication factor
- To define a storage policy (e.g. a "performance" tier with RF=3 and encryption on) that pools or workloads should conform to
- To watch a VM disk migration between pools and confirm it completed
- To group pools into a datastore cluster that rebalances automatically once a space or IO-latency threshold is crossed

How to get there

- Route / id: `/distributed-storage`
- Nav: **Infrastructure** → **Distributed Storage** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Four tabs — **Pools**, **Policies**, **Migrations**, **Datastore Clusters** — plus summary cards for total pools, aggregate capacity, policy count, and active migrations.

1. **Pools** — distributed storage pools with a status badge, host count, and replication factor, and a used/available capacity bar. **Create Pool** sets a name, type, and replication factor. Delete a pool with a confirmation dialog.
2. **Policies** — a table of storage policies showing tier, replication factor, failure tolerance, and whether encryption, deduplication, and compression are on. **Create Policy** sets name, description, replication factor, stripe width, failures-to-tolerate, and tier (performance / standard / archive). Delete with confirmation.
3. **Migrations** — a read-only table of VM storage migrations: source pool, target pool, progress %, bytes transferred, and status (pending / in progress / completed / failed).
4. **Datastore Clusters** — a table of clusters showing datastore count, whether storage DRS (SDRS) is enabled, space threshold %, total capacity, and VM count. **Create Cluster** sets a name, space threshold %, and IO-latency threshold (ms) that trigger automatic rebalancing.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Net Security

Purpose

Net Security — the advanced SDN and security control plane: network policies scoped to security identities, host firewall profiles/zones/VM assignments, exposed services, QoS traffic shaping, DNS zones/policies, WireGuard VPN tunnels and networks, traffic mirroring, NAT rules/pools/gateways, and bandwidth monitoring with alerts. For everyday per-VM networking mode and port forwards, see [Network](#) instead.

When to use it

- To write a network policy scoped to a security identity (a label-based group of VMs) rather than to individual IPs
- To manage host firewall zones/profiles and see which VMs are assigned to which firewall profile
- To set up a WireGuard VPN tunnel or network between sites
- To rate-limit a VM's traffic with a QoS policy, or mirror its traffic to a collector for inspection
- To bring firewalld/nftables/WireGuard configuration that already exists on the host under Zylor's management instead of recreating it by hand

How to get there

- Route / id: `/app/network-security`
- Nav: **Infrastructure** → **Net Security** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Nine tabs, each with its own live count in the header: **Policies**, **Firewall**, **Services**, **QoS**, **DNS**, **VPN**, **Mirror**, **NAT**, **Monitor**. Data refreshes automatically every 15 seconds and on demand via **Refresh**. Read-only accounts see a read-only notice and can't create, adopt, sync, or delete anything.

1. **Policies** — network policies bound to security identities (identities are auto-created from VM labels or discovered from firewalld zones/nftables sets on the host). Create or delete a policy; adopt a discovered policy or identity into management.
2. **Firewall** — firewall profiles, zones, and per-VM assignments. Create/delete profiles and zones, remove a VM's firewall assignment, and adopt a zone or profile that already exists on the host.
3. **Services** — exposed host listeners. Create/delete a service, or adopt one already running on the host.
4. **QoS** — bandwidth-shaping policies. Create/delete/adopt.
5. **DNS** — DNS zones and DNS policies. Create/delete/adopt both zones and policies.
6. **VPN** — WireGuard tunnels and VPN networks. Create/delete tunnels and networks; adopt a tunnel already configured on the host.
7. **Mirror** — traffic mirror sessions that copy a VM's traffic to a collector address. Create/delete/adopt.
8. **NAT** — NAT rules, NAT pools, and gateway configs. Create/delete each; adopt a NAT rule found on the host.
9. **Monitor** — bandwidth monitor policies, live traffic metrics, and bandwidth alerts. Create/delete/adopt a monitor policy, and acknowledge an alert.

Every tab has a **Sync** action that rescans the host's actual configuration (firewalld, nftables, WireGuard, etc.) for items not yet under Zylor's management — synced items show up as adoptable so you can bring them under management with one click instead of recreating them.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Network

Purpose

Network — day-to-day VM networking: which mode a VM uses, its port forwards, and its assigned address. For the advanced SDN stack (policies, firewalls, VPN mesh, QoS, mirroring), see [Net Security](#) instead.

When to use it

- To see how a VM is networked and what's reachable from outside the host
- To add or remove a port forward without recreating the VM
- To check a VM's assigned IP address

How to get there

- Route / id: `/app/network`
- Nav: **Infrastructure** → **Network** (sidebar, command palette, or desktop nav)
- Per-VM networking is also managed from that VM's own **Network** tab on its detail page

Operate from the console (UX)

1. Review VMs by networking mode:
 - **NAT** — private, outbound-only by default; individual ports can be exposed with a host→guest port forward (set at creation in the [Create VM](#) wizard, or added later from a VM's Network tab).
 - **Bridged** — the VM has its own address on the local network, either DHCP-assigned or set statically via cloud-init at boot.
2. Add or remove a port forward for a running or stopped VM — if the VM is running, it restarts automatically to apply the change.
3. Confirm a forwarded port is actually reachable from another machine on your network, not just from the host itself.
4. Check a bridged VM's assigned IP address once it's leased (or immediately, if it was assigned statically).

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Net Security](#)
 - [Create VM](#)
 - [Virtual Machines](#)
 - [Getting Started](#)
 - [Page index](#)
-

Resource Pools

Purpose

Resource Pools — hierarchical CPU/memory allocation pools (nested, with shares, reservations, and limits) that VMs draw from, plus an admission-control test to check whether a workload's requirements would fit before you commit to it.

When to use it

- To divide a cluster's CPU/memory into weighted pools — for example giving "prod" more shares than "dev"
- To nest a child pool under a parent to further subdivide its allocation
- To check whether a pool has room for a new VM's CPU/memory requirements before creating it
- To see how much of a pool's reservation or limit is actually in use

How to get there

- Route / id: `/resource-pools`
- Nav: **Infrastructure** → **Resource Pools** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review stat cards: total pools, total CPU shares, and total VMs across all pools.
2. Browse the expandable pool tree — click a pool to expand/collapse its children; each row shows VM count, CPU/memory shares, and live usage bars that turn red above 80%.
3. Expand a pool to see its CPU/memory reservation, limit, currently available capacity, and child pool count.
4. **Create Pool** — set a name, cluster ID, an optional parent pool (to nest it under another pool), and CPU/memory shares, reservations, and limits (use `-1` for unlimited).
5. **Test Admission** — on any pool, enter a required CPU (MHz) and memory (MB) and run the check; the result shows Admitted or Denied with a reason and the pool's currently available capacity.
6. Delete a pool — asks for confirmation before removing it.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Storage Pools

Purpose

Storage Pools — create, start/stop, and monitor the storage backends VM disks live on: local directories, NFS exports, LVM and LVM-thin volume groups, ZFS pools, or Ceph RBD pools. This is where a pool's lifecycle and health live; for the volumes (disks) inside those pools see [Storage](#), and for multi-host replicated/policy-driven storage see [Distributed Storage](#).

When to use it

- To add a new storage backend — e.g. mount an NFS export or attach a Ceph RBD pool — for VM disks to be created on
- To start or stop a pool, or delete one that's no longer needed
- To check NFS or Ceph pool health at a glance
- To search for a pool by name, path, or state and see its live capacity/usage

How to get there

- Route / id: `/app/storage-pools`
- Nav: **Infrastructure** → **Storage Pools** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review stat cards: total pools, active pools, total capacity, and total available space.
2. Search pools by name, path, or state.
3. Table of pools — name, type with backend-specific detail (NFS server:export path, LVM volume group, LVM-thin volume group/thin pool, ZFS zpool/dataset, or Ceph pool name + monitor count), path, capacity, available space, a usage bar, current state, and a health indicator for NFS/Ceph pools.
4. **Create Pool** — pick a type (Local, NFS, LVM, LVM-thin, ZFS, Ceph) and fill in its backend-specific fields (for NFS: server, export path, mount path, NFS version, mount options; for Ceph: monitor addresses, pool name, optional user/keyring; etc.), plus an auto-start-on-daemon-boot toggle.
5. Per-pool actions: **Start** an inactive pool, **Stop** an active one, **Refresh** to pull current stats, or **Delete** — disabled while the pool is active, so stop it first — with a confirmation dialog.
6. For Ceph pools, a **Manage RBD images** action opens a panel to list, create, and delete raw RBD images in that pool. This is proxied through the Atlas storage control plane (Zyvor's Ceph system of record) and requires Atlas to be configured on the backend; image creation is queued and completes asynchronously rather than immediately.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed. Ceph pool health/stats and RBD image management additionally require an Atlas connection — if that's not configured, those specific actions will show an unavailable error rather than the whole page failing.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Storage

Purpose

Storage — a consolidated view of every storage pool's capacity alongside a manual volume tracking ledger. To create or manage a pool itself, use [Storage Pools](#); for replicated/policy-driven storage across hosts, see [Distributed Storage](#).

When to use it

- To see total capacity, used space, and pool count across the whole host in one place
- To keep a manual record of volumes you've provisioned elsewhere (which pool, what size, which VM it's meant for) alongside their real-world lifecycle

How to get there

- Route / id: `/app/storage`
- Nav: **Infrastructure** → **Storage** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review stat cards: total capacity, used space, volume count, and pool count.
2. Storage Pools panel — each pool's name, path, type badge, state, a used/total usage bar color-coded by how full it is, and an **Add Volume Record** button.
3. Volumes table — every volume record across all pools: name, pool, size, the VM it's marked attached to (or "Not attached"), and **Resize**, **Attach/Detach**, and **Delete** actions (delete has a confirmation dialog).

Important: the Volumes table is a manual tracking ledger, not live disk management — creating, resizing, attaching, or detaching a record here only updates that record. It does not provision, resize, or attach a real disk image, and does not change any VM's actual configuration. Use it to keep notes on volumes you provision through other means (e.g. directly on the pool, or via Storage Pools' RBD image management for Ceph). Pool creation and lifecycle (start/stop) live on [Storage Pools](#).

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

System Health

Purpose

System Health — a live, read-only dashboard of host resource utilization, refreshing every 2 seconds: CPU, memory, disk I/O, filesystems, network interfaces, and top processes, rolled up into a single health score.

When to use it

- To get a fast, single-number (0–100) read on whether the host is under stress
- To find out what's bottlenecking the host — CPU, memory, disk, or network — before adding more VMs to it
- To identify the specific process consuming the most CPU or memory
- To check filesystem usage or per-disk I/O latency and queue depth on the physical host

How to get there

- Route / id: `/app/system-health`
- Nav: **Infrastructure** → **System Health** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Health score gauge (0–100, colored red/amber/green) with a status label and a summary panel calling out the current bottleneck, if any.
2. **CPU** panel — overall usage, core count, 1-minute and 5-minute load averages, and a per-core usage grid.
3. **Memory** panel — RAM and swap usage bars with used/total detail, plus total, available, and cached memory.
4. **Filesystems** — a usage bar per mounted filesystem (mountpoint, filesystem type, used/total), shown when data is available.
5. **Disk I/O** — per-device reads/writes completed, bytes read/written, average latency, and queue depth.
6. **Network** — per-interface RX/TX bytes and error counts, TCP connection-state counts, and host-wide RX/TX/retransmit totals.
7. **Top CPU** and **Top Memory** process tables — PID, name, CPU%, and memory (MB).

Entirely read-only monitoring — there's no create/edit/delete action here; data refreshes automatically every 2 seconds.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

System

Purpose

System — the physical host's hardware topology (CPU sockets/cores/threads, NUMA nodes, hugepages) plus topology-aware optimization recommendations for individual VMs. This is distinct from [System Health](#), which tracks live utilization rather than hardware layout.

When to use it

- To see the host's CPU socket/core/thread layout and NUMA node boundaries before pinning a VM's vCPUs
- To check how many hugepages (2MB/1GB) are allocated and free, or allocate more for a memory-intensive VM
- To review and apply topology-aware optimization recommendations (e.g. NUMA/CPU pinning) for a specific running VM

How to get there

- Route / id: `/app/system`
- Nav: **Infrastructure** → **System** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Four tabs — **CPU Topology**, **NUMA Topology**, **Memory & Hugepages**, **Optimization** (badged with the pending recommendation count) — plus stat cards for total CPUs (sockets × cores), NUMA node count, total/available memory, and 2MB hugepage total/free.

1. **CPU Topology** — CPUs grouped by socket, showing each CPU's online/offline status; hover a CPU for its core, thread, and NUMA node.
2. **NUMA Topology** — per-node CPU list, memory total/free with a usage bar, per-node hugepage counts, and an inter-node distance matrix.
3. **Memory & Hugepages** — system memory breakdown (total, available, usage bar, buffers, cached) plus 2MB and 1GB hugepage stats. **Allocate Hugepages** opens a dialog to pick a page size and count and apply it immediately.
4. **Optimization** — per-VM recommendations (resource, current → recommended value, reason, expected impact). **Apply** runs the recommended change against that VM right away.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

VM Pools

Purpose

VM Pools — warm / capacity pools for fast provisioning.

When to use it

- Open this surface when the job matches the purpose above
- Start from the product home / dashboard if you are unsure where to begin
- Confirm auth and that required backends/operators are reachable if data looks empty

How to get there

- Route / id: `/app/vm-pools`
- Nav: **Infrastructure** → **VM Pools** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Open `/app/vm-pools` and wait for live data from Zyvor Fabric.
2. Use filters and search when the page provides them.
3. Drill into a row or card for detail, then jump to related surfaces.
4. For mutating actions: review impact, role gates, and confirmation dialogs first.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Zones

Purpose

Zones — Infrastructure surface for placement domains.

When to use it

- Open this surface when the job matches the purpose above
- Start from the product home / dashboard if you are unsure where to begin
- Confirm auth and that required backends/operators are reachable if data looks empty

How to get there

- Route / id: `/zones`
- Nav: **Infrastructure** → **Zones** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Open `/zones` and wait for live data from Zyvor Fabric.
2. Use filters and search when the page provides them.
3. Drill into a row or card for detail, then jump to related surfaces.
4. For mutating actions: review impact, role gates, and confirmation dialogs first.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Alerts

Purpose

Alerts — a live view of currently firing system alerts and the notification rules that generate them, polling for updates automatically.

When to use it

- To see at a glance whether anything is critical or warning-level right now
- To check what an active alert actually means before deciding whether to act
- To review which alert rules are configured and enabled, and at what threshold they fire

How to get there

- Route / id: `/app/alerts`
- Nav: **Monitoring** → **Alerts** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Active Alerts summary** — three stat tiles at the top show the total number of active alerts, how many are critical, and how many are warning-level.
2. **Active Alerts list** — each alert is a card showing its severity badge (critical/warning/info), timestamp, title, message, and the triggering value when one is available. Cards are left-bordered in red, amber, or blue to match severity. If nothing is currently firing, the list shows "No active alerts".
3. **Alert Rules table** — when rules exist, a table lists each rule's name, condition, threshold, severity, and whether it's enabled (Yes/No). This is a read-only view of what's configured, not an editor.
4. The page refreshes itself every 5 seconds in the background; if a background refresh fails, a small amber banner notes it and continues showing the last known data rather than clearing the screen. Use the header's refresh control to force an immediate reload.

This page is read-only — there's no way to acknowledge, mute, or create alerts/rules from here.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Analytics

Purpose

Performance Analytics — fleet-wide resource utilization, trends over time, and per-VM performance insights, with exportable reports.

When to use it

- To check overall CPU/memory/disk/network utilization across the fleet
- To find which VMs are consuming the most CPU, memory, or network right now
- To review flagged performance issues (e.g. a VM running hot on a resource) along with a recommendation
- To pull a PDF or CSV performance report for a stakeholder or an incident writeup

How to get there

- Route / id: `/app/analytics`
- Nav: **Monitoring** → **Analytics** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Time range** — pick Last Hour, 6 Hours, 24 Hours, 7 Days, or 30 Days from the dropdown; it drives the performance chart and reloads all data on the page.
2. **Export** — the Export button opens a menu to download a performance report as PDF or CSV for the selected time range.
3. **Resource Utilization Overview** — four tiles (CPU, Memory, Disk, Network) show current utilization percentage with a color-coded bar (green under 50%, blue 50-75%, amber 75-90%, red 90%+).
4. **Performance Insights** — up to 5 flagged issues, each tagged critical/warning/info, naming the VM, the resource, its current value, and a written recommendation.
5. **Top VMs by Resource** — three ranked lists (by CPU, by Memory, by Network) showing the top 5 VMs each, with a bar and percentage (or Mbps for network).
6. **System Performance Over Time** — an area chart of total CPU% and memory% across the selected time range, plus summary figures below it: average CPU, average memory, total VMs, and running VMs for that window.

This page is read-only — it reports on performance, it doesn't let you act on a VM directly (use the VM detail page for that).

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Audit

Purpose

Audit Logs — the security and compliance trail of who did what: every tracked action, which user performed it, on which resource, whether it succeeded, and from what IP address.

When to use it

- To investigate who created, deleted, started, or stopped a specific VM (or other resource) and when
- To check recent failures for signs of misconfiguration or unauthorized attempts
- To pull an export of the audit trail for a compliance review or incident report

How to get there

- Route / id: `/app/audit`
- Nav: **Monitoring** → **Audit** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Stats row** — total logs, success rate, recent failures, and the top 3 most common actions.
2. **Search** — free-text search box matches against action, user, resource name, and resource type; press Enter or the results filter live as you type.
3. **Filters** — toggle the Filters panel to narrow by Status (success/failed) and Resource Type (VM, network, storage, template, quota, schedule); **Clear Filters** resets both the filters and the search box.
4. **Export** — the Export button downloads the currently filtered log set as JSON or CSV.
5. **Logs table** — each row shows a relative timestamp, the acting user, the action (color-coded: green for create/start, red for delete/stop, amber for update/edit), the resource type and name, a success/failed status badge (with the error message inline if it failed), and the source IP address. The footer shows how many of the total logs match the current filters.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Capacity

Purpose

Capacity Planning — resource usage against total capacity per resource (memory, CPU, storage), with week-over-week trend, so you can see what's running out and when.

When to use it

- To check whether the host is approaching a resource ceiling before it becomes an incident
- To see which resources are growing week over week and by how much
- To get a rough sense of how many active VMs are contributing to current usage

How to get there

- Route / id: `/capacity-planning`
- Nav: **Monitoring** → **Capacity** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Capacity warning banner** — appears automatically whenever any tracked resource is above 75% usage, naming which ones.
2. **Summary tiles** — Active VMs (count), Resources Tracked, resources Over 75% Usage, and Growing Resources (trending upward).
3. **Per-resource cards** — one card per tracked resource (e.g. Memory, CPU Cores, Storage), each showing: an icon, used/total with units (auto-converts GB to TB past 1024 GB), current usage percentage, a color-coded progress bar (green/blue/amber/red by thresholds), the weekly trend (%/week, with an up/down arrow), how much capacity remains, and a projected-full date when the backend supplies one.
4. The page refreshes itself every 30 seconds; use the header refresh control to force an immediate reload.

This page is read-only reporting — there's nothing to configure or act on here directly.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Debug Tools

Purpose

Debug Tools — raw, terminal-style output from four classic Linux diagnostic commands (top, iostat, vmstat, netstat) against the host, rendered as monospace panels. This is the closest the dashboard gets to SSHing in and running commands yourself.

When to use it

- To check live process/CPU activity, disk I/O, virtual memory stats, or network connections on the host without opening a shell
- To troubleshoot a performance problem in real time, side by side across all four views
- When Kernel or Analytics show something is wrong and you need the raw command output to confirm it

How to get there

- Route / id: `/debug`
- Nav: **Monitoring** → **Debug Tools** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Four independent panels** — Top, IOStat, VMStat, and NetStat, each backed by its own endpoint and showing raw output lines in a scrollable monospace box.
2. Panels don't load automatically — each starts with "Click Refresh to load data" until you refresh it (individually, via the **Refresh** button in that panel's header) or use **Refresh All** in the page header to load all four at once.
3. **Auto-refresh** — toggle the switch in the header to re-pull all four panels every 3 seconds; useful for watching a metric change live. Turn it off when you're done to stop the polling.
4. If a panel's fetch fails, that panel shows its own error message rather than blocking the others.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Event Stream

Purpose

Event Stream — a live, scrolling log of VM lifecycle events (create, start, stop, delete, and similar) pushed over an authenticated SSE connection as they happen. There's no history — you only see events that occur while the page is open.

When to use it

- To watch VM lifecycle activity happen in real time, e.g. while running a script that creates or tears down several VMs
- To confirm an action you just took (start, stop, delete) actually registered
- To catch error/warning-level events as they occur without polling a log page

How to get there

- Route / id: `/event-stream`
- Nav: **Monitoring** → **Event Stream** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Connection indicator** — shows Connected (green) or Reconnecting... (amber, pulsing) for the underlying SSE connection.
2. **Pause / Resume** — pause freezes the view and drops incoming events instead of queuing them; resume starts appending new events again from that point on.
3. **Clear** — empties the current list of events shown (does not affect the connection).
4. **Level filter** — narrow the view to All, Info, Warning, Error, or Debug; level is inferred from the event type (e.g. an event type containing "fail" or "error" shows as Error).
5. Each line shows the time, level, source VM name, and message. The stream keeps the most recent 500 events and auto-scrolls to the newest as they arrive (unless paused). If nothing is happening on the fleet, it simply reads "Waiting for events..." — that's expected, not a failure.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Explain

Purpose

Explain — plain-language, AI-generated explanations for a chosen system metric: its current value and trend, an assessment of its status, what's contributing to it, and what to do about it. This is the interpretive layer on top of the raw numbers you'd see in Analytics or Debug Tools.

When to use it

- When a metric looks off and you want a written explanation of what's driving it, not just the raw number
- To get concrete recommendations for a specific resource (CPU, memory, disk, or network) instead of interpreting a chart yourself
- Before escalating an issue, to see whether the system already has a plausible explanation and fix

How to get there

- Route / id: `/explain`
- Nav: **Monitoring** → **Explain** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Pick a metric** — four buttons: CPU, Memory, Disk, Network. Selecting one fetches its explanation and its last-hour timeseries; nothing loads until you pick one.
2. **Current Value & Status** — shows the metric's current value with a trend arrow (up/down/flat) and a status badge (e.g. normal, elevated, critical) with a short written summary.
3. **Last Hour chart** — a simple bar chart of the metric's samples over the past hour; hover a bar to see its exact value and time.
4. **Contributing Factors** — a grid of named factors, each with an impact badge (high/medium/low) and a short description of how it's affecting the metric.
5. **Recommendations** — a checklist of suggested actions to address the metric, when the backend provides any.

This page is read-only and doesn't take any action on your behalf — it only explains and recommends.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Kernel

Purpose

Kernel — a snapshot of the host's kernel configuration: version, hostname, architecture, boot command line, loaded kernel modules, and sysctl parameters. This is static configuration, not live activity — for that, see Debug Tools or Event Stream.

When to use it

- To confirm what kernel version, architecture, or boot parameters the host is running
- To check whether a specific kernel module is loaded and what's using it
- To look up the current value of a sysctl parameter without shelling in

How to get there

- Route / id: `/kernel`
- Nav: **Monitoring** → **Kernel** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Summary tiles** — Kernel Version, Hostname, Architecture, and Modules Loaded count.
2. **Boot Command Line** — shown verbatim in a code block when the backend reports one.
3. **Kernel Modules table** — name, size, and "used by" (dependent modules) for each loaded module, capped at the first 100 matching rows. A **Filter modules** box narrows the table live by module name.
4. **Sysctl Parameters table** — key/value pairs for the sysctl settings the backend reports, shown in full (no filter).
5. The page refreshes itself every 10 seconds; use the header refresh control to force an immediate reload. This page is read-only — there's no way to load a module or change a sysctl value from here.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Live Metrics

Purpose

Live Metrics — a real-time view of host performance: CPU, memory, disk I/O, and network throughput, each as a rolling sparkline that updates once per second.

When to use it

- Watching load in real time while you run a benchmark or try to reproduce a slow VM
- Confirming the host is actually under CPU, memory, disk, or network pressure right now
- Pausing the feed to freeze a spike so you can read the exact numbers or take a screenshot

How to get there

- Route / id: `/app/live-metrics`
- Nav: **Monitoring** → **Live Metrics** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

- Six live cards — CPU Usage, Memory Usage, Disk Read, Disk Write, Network RX, Network TX — each showing the current value and a 60-second sparkline history, polled every second.
- Network RX/TX are computed rates (bytes/sec), derived on the client from the delta between successive cumulative counter samples — not raw totals.
- **Pause / Resume** — freezes or resumes the 1-second polling loop; useful for holding a spike still to inspect it.
- A status dot next to Pause/Resume shows **Streaming** (green, pulsing), **Paused** (amber), or **Error** (red).
- If a background refresh fails after data has already loaded, an amber banner reports it while continuing to show the last known values instead of blanking the page.
- If the first load fails, an error banner replaces the cards with retry hints.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Logs

Purpose

Logs — a searchable, filterable console view of Zylvor Fabric's audit log: every recorded action and event, level-coded and continuously refreshed.

When to use it

- Investigating what changed and when — VM creates/deletes, config changes, and other recorded actions
- Filtering down to just `ERROR` / `WARN` entries to find what went wrong
- Searching by keyword or source, or exporting a filtered slice, before opening a support ticket

How to get there

- Route / id: `/app/logs`
- Nav: **Monitoring** → **Logs** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Filter box** — matches against the log message and source as you type.
2. **Level dropdown** — narrow to `ALL`, `INFO`, `WARN`, `ERROR`, or `DEBUG`.
3. **Auto-scroll** checkbox — keeps the view pinned to the newest entry as new ones arrive (the feed polls every 5 seconds).
4. **Refresh** — manual reload in addition to the automatic poll.
5. **Export** — downloads the currently filtered entries as a plain-text `.txt` file (timestamp, level, source, message).
6. **Clear** (trash icon) — clears the log view locally only; it doesn't delete the underlying audit history, so the entries reappear on the next refresh or poll.
7. Each entry is color-coded by level (cyan info, yellow warn, red error/critical, slate debug) and shows timestamp, level, source, and message.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Notifications

Purpose

Notifications — configure how and when Zylvor Fabric alerts you: the delivery channels (email, Slack, Teams, webhook), the rules that trigger them, and a history of what was actually sent.

When to use it

- Setting up a Slack, Teams, email, or webhook channel to receive alerts
- Checking whether a rule has fired, how often, and whether delivery succeeded or failed
- Testing a channel to confirm it's wired up correctly before relying on it
- Temporarily disabling a rule or channel without deleting its configuration

How to get there

- Route / id: `/notifications`
- Nav: **Monitoring** → **Notifications** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

The page has three tabs — **Channels**, **Rules**, **History** — loaded together on open.

Channels (default tab)

- **Add Channel** opens a form: name, type (Webhook, Slack, Teams, Email), and type-specific fields — a URL for Webhook/Slack/Teams, or SMTP host/port + from/to address for Email.
- **Test** sends a real test notification through that channel (disabled if the channel isn't enabled).
- **Delete** removes a channel after a confirmation prompt.

Rules

- Each rule shows enabled/disabled state, its description, up to 3 event-type tags (with a "+N more" overflow), how many times it has triggered, and when it last fired.
- The enable/disable toggle (power icon) flips a rule on or off immediately.
- **Delete** removes a rule after a confirmation prompt.
- **Create Rule** opens a form: name, description, VM event types (created, started, stopped, snapshot taken, hotplug, error, etc.), severity levels, and which channels to notify. Every enabled rule is evaluated in real time against actual VM events — a matching rule sends through each selected channel and updates its trigger count and last-fired time here.

History

- A table of everything that's actually been sent: timestamp, rule name, event type (severity-colored), affected VM (if any), channel, and delivery status (`SENT` / `FAILED`).

If the channels list fails to load, an error banner with retry appears; rules and history failing to load independently just show as empty rather than blocking the page.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)

- [Page index](#)
-

Processes

Purpose

Processes — a live process monitor for the host: every OS process with its CPU and memory usage, refreshed every 3 seconds, with a per-process detail drill-down.

When to use it

- Finding what's consuming CPU or memory on the host right now
- Spotting zombie or stopped processes, or an unexpectedly high process count
- Digging into a specific process's command line, I/O, open file descriptors, or context-switch activity

How to get there

- Route / id: `/processes`
- Nav: **Monitoring** → **Processes** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Summary cards show **Total Processes**, **Running**, and **Sleeping** counts.
2. Filter box matches by PID, process name, or state as you type.
3. The process table lists PID, name, CPU % (with a color bar — green under 20%, blue under 50%, amber under 80%, red at 80%+), memory in MB, a state badge (Running / Sleeping / Disk Wait / Zombie / Stopped), and thread count.
4. Click any row to open a **Process Detail** panel below the table — fetched per-PID — showing command line, IO read/write bytes, open file descriptors, and voluntary/involuntary context switches. **Close** collapses it.
5. The list auto-refreshes every 3 seconds; there's also a manual refresh button in the page header.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Optimizer

Purpose

Optimizer — a right-sizing advisor that analyzes each VM's actual resource usage and recommends CPU/memory/disk adjustments, with a one-click apply per VM.

When to use it

- Checking whether VMs are over- or under-provisioned before a capacity or cost review
- Applying a recommended resource change without manually editing a VM's spec
- Triaging which VMs have high-impact recommendations first

How to get there

- Route / id: `/resource-optimizer`
- Nav: **Monitoring** → **Optimizer** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Summary cards show **VMs Analyzed**, total **Recommendations**, **High Impact**, and **Medium Impact** counts.
2. If nothing needs changing, the page shows an "All VMs are optimally configured" state instead of a list.
3. Otherwise, one card per VM lists its recommendations: the affected resource, current value → recommended value, the reason, and an impact badge (High / Medium / Low, color- and icon-coded).
4. **Auto-Optimize** per VM applies all of that VM's recommendations via a single call to the VM's optimize endpoint — the button spins while applying, then becomes a disabled "Applied" state and reports how many changes were applied vs. skipped.
5. Manual refresh is available from the page header.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Service Map

Purpose

Service Map — shows the services discovered across your VMs, which ones depend on each other (protocol and port), and each service's current health.

When to use it

- Understanding what a VM's service depends on, or what depends on it, before restarting or changing it
- Spotting degraded or down services at a glance
- Tracing a specific inbound or outbound connection between two services

How to get there

- Route / id: `/service-map`
- Nav: **Monitoring** → **Service Map** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Summary cards show **Services**, **Healthy**, **Degraded**, and **Down** counts.
2. Service cards in a grid show name, a health dot (green/amber/red/gray), a type badge (web, database, cache, queue, api, proxy, ...), the host VM, port, and inbound/outbound link counts.
3. Click a service card to filter the **Dependencies** list down to just that service's links (both directions); unrelated cards dim while one is selected. **Show all** clears the selection.
4. The Dependencies panel lists each link as `From → To` with its protocol and port.
5. The map refreshes automatically every 15 seconds; a manual refresh button is also in the page header.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Timeline

Purpose

Timeline — a single reverse-chronological activity feed that merges audit-log actions and system alerts, so you can see what happened and in what order without switching between Logs and Notifications.

When to use it

- Reconstructing a sequence of events — what led up to an error, in order
- Getting a quick "what's happened lately" view of the whole system
- Filtering down to just deploys, or just errors, to review one class of events

How to get there

- Route / id: `/timeline`
- Nav: **Monitoring** → **Timeline** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Filter chips — **All, Actions, Alerts, Deploys, Errors** — filter the merged feed client-side.
2. Each entry is auto-classified and shown with an icon/color, description, relative timestamp, and type tag: audit entries with a failed/error status become **Error**, create/deploy-style actions become **Deploy**, other audit entries become **Action**; alerts become **Alert**, or **Error** if their severity is critical/error.
3. The feed auto-refreshes every 10 seconds; the header shows "Updated Xm ago," and there's also a manual refresh button.
4. If a background refresh fails after data has already loaded, an amber banner reports it while the last known feed stays on screen.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Backup Scheduler

Purpose

Backup Scheduler — create and manage recurring, automated backup jobs that snapshot one or more VMs' disks to a directory on a schedule, with retention and format controls.

When to use it

- To set up nightly or weekly backups for a group of VMs instead of exporting disks by hand
- To back up several VMs on a shared cadence (e.g. every 6 hours) with a single schedule
- To control how much backup history is kept, and in what disk format, without babysitting the job

How to get there

- Route / id: `/backup-scheduler`
- Nav: **More — images, migrations & managers → Backup Scheduler** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Select VMs — check the VMs you want the schedule to cover (each row shows its current state, e.g. `running`);

Select All / Deselect All toggles the whole list at once.

Schedule Configuration — build the job:

- **Schedule Name** — a label for the job (e.g. `nightly-backup`).
- **Frequency** — pick a preset (**Daily 2 AM**, **Weekly Sun 3 AM**, **Every 6h**) or **Custom** to enter a raw cron expression; the page shows a plain-English readout of what the cron actually means (e.g. "Daily at 2:00 AM").
- **Output Directory** — where backups land on the host (defaults to `/var/lib/zyvor-fabricd/backups`).
- **Retention (keep last N)** — how many backups to retain before older ones are pruned (1–365).
- **Format** — QCOW2, RAW, or VMDK for the backup disk files.
- **Enable compression** — toggle to compress backup output.

Click **Create Schedule** to save it — you need at least one VM selected and a schedule name, or the page shows a validation error inline.

Existing Schedules — lists every configured schedule with its enabled/disabled state, cron expression (raw and human-readable), how many VMs it covers, and its next run time when available.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Batch Import

Purpose

Batch Import — bulk-create VMs from a YAML or JSON list, with a preview step and a per-VM status readout as each one is submitted.

When to use it

- To stand up several VMs at once (e.g. `web-01` , `db-01` , `app-01`) from a single definition file instead of using the Create VM wizard repeatedly
- To reprovision from a saved YAML/JSON inventory of VMs — one you maintain outside the product (a script, a checked-in file) — starting from the downloadable template as a model
- To spot bad entries (missing name or image) before anything is created, and see exactly which VMs succeeded or failed if some fail

How to get there

- Route / id: `/batch-import`
- Nav: **More — images, migrations & managers → Batch Import** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Provide the input** — drag and drop a `.yaml` / `.yml` / `.json` file onto the drop zone (or click to browse), or paste YAML/JSON directly into the text box. Each entry needs a `name` and `image` ; `cpus` and `memory` are optional and default to `2` and `2G` . Click **Download Template** for an example file with three sample VMs.
2. **Preview** — click **Preview** to parse the input into a table of VMs. If entries are missing a name or image, or nothing parses, you get an inline error instead of moving on.
3. **Review the table** — each row shows the VM's name, vCPUs, memory, and image path, plus a status icon (pending, submitting, submitted, or failed).
4. **Submit All** — creates the VMs one at a time via the VM creation API. Each row's status updates live as it's submitted; failed rows show the specific error message under the image path. A summary line at the top tracks total, submitted, and failed counts.
5. **Back to Editor** — return to the input step to fix and re-preview without losing your place.
6. **Empty / fail:** Check health, auth, and domain dependencies.
7. **Success:** Live data loads; mutations complete without error toasts.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Batch Migration

Purpose

Batch Migration Builder — a form-based editor for assembling a multi-VM migration job spec (source disk, target format, sizing) and exporting it as JSON. It builds the spec only; it does not run the migration itself.

When to use it

- To describe several VMs' migrations (source disk path, target format, vCPUs, memory) in one place before handing the job to a migration tool or pipeline
- To generate a JSON migration manifest you can copy into another tool, save as a file, or version-control
- To sketch out a batch of conversions (e.g. from VMDK to QCOW2) without touching a text editor

How to get there

- Route / id: `/batch-migration`
- Nav: **More — images, migrations & managers** → **Batch Migration** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Add VM** — adds a new, expanded entry card to the list; each is numbered in order.
2. **Fill in each entry** — VM Name, Source Path (e.g. `/path/to/disk.vmdk`), Target Format (QCOW2, RAW, or VMDK), vCPUs, and Memory (MB). Click an entry's header to collapse or expand it; the collapsed view still shows the name and source path for quick scanning.
3. **Remove** — the trash icon on an entry deletes it from the list.
4. **JSON Preview** — as soon as you have at least one entry, a live JSON preview builds the migration spec (`migrations: [...]` with `vm_name`, `source_path`, `target_format`, `cpus`, `memory_mb` per entry).
5. **Copy** or **Download** the generated JSON to use elsewhere — Copy puts it on the clipboard (with a brief confirmation), Download saves it as `batch-migration.json`.
6. **Empty / fail:** Check health, auth, and domain dependencies.
7. **Success:** Live data loads; mutations complete without error toasts.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Disk Converter

Purpose

Disk Format Converter — convert a single disk image between QCOW2, VMDK, VHD, VHDX, and RAW, tracking the conversion job's progress to completion.

When to use it

- To convert an imported disk (e.g. a VMDK from another hypervisor) into QCOW2 before using it with a VM
- To produce a RAW or VHD/VHDX copy of a disk for a tool or target platform that needs a specific format
- To watch a long-running conversion through to completion without leaving the page

How to get there

- Route / id: `/disk-converter`
- Nav: **More — images, migrations & managers → Disk Converter** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Pick a source disk** — choose from the dropdown of disk images already known to the host, or type a path directly (e.g. `/path/to/disk.vmdk`). If no images are listed, click **Load available disk images** to fetch them.
2. **Choose the target format** — QCOW2, VMDK, VHD, VHDX, or RAW.
3. **Output path** — auto-derived from the source path and target format (e.g. `disk.qcow2`); edit it manually if you want a different destination.
4. **Convert** — submits the job and starts polling its status every 2 seconds. A progress bar and percentage track it live through pending → running → completed/failed.
5. On failure, the error message from the backend is shown inline; on success, the output path is displayed.
Reset clears the form and job state to start over.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Disk Images

Purpose

Disk Images — a read-only inventory of the VM disk images present on the host: name, format, size, and path for each one.

When to use it

- To see at a glance how many disk images exist on the host, their total footprint, and which formats are in use
- To find a specific image by name, format, or path before referencing it elsewhere (e.g. when creating a VM from a custom disk path)
- To select one or more images as a quick visual tally before acting on them from another page

How to get there

- Route / id: `/disk-images`
- Nav: **More — images, migrations & managers** → **Disk Images** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

- Review summary tiles for **Total Images**, **Total Size**, distinct **Formats**, and how many rows you currently have **Selected**.
- **Search** by name, format, or path to narrow the table.
- Click a row (or its checkbox) to select/deselect it — selection is local to the page and just feeds the Selected counter; there's no bulk action attached to it here.
- Each row shows the image's name, a color-coded format badge (qcow2, vmdk, vhd/vhdx, raw, img), size, and full path.
- **Refresh** reloads the list from the host.

5. **Empty / fail:** Check health, auth, and domain dependencies.

6. **Success:** Live data loads; mutations complete without error toasts.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Download Disk

Purpose

Download Disk — browse the disk images available on the Fabric host and download any of them straight to your machine.

When to use it

- To pull a copy of a VM disk image off the host — for backup, for inspection, or to move it to another environment
- To check how much disk-image storage a host is using and how many distinct formats are present
- To locate an image that lives outside the default images directory by browsing a custom path

How to get there

- Route / id: `/download-disk`
- Nav: **More — images, migrations & managers** → **Download Disk** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

- Review summary tiles for **Total Images**, **Total Size**, and how many distinct **Formats** are present (qcow2, vmdk, vhd/vhdx, raw, img — each gets its own color badge in the table).
 - **Filter** the list by name, format, or path, and **sort** by Name, Size, or Modified (click a column header to sort, click again to reverse direction).
 - Each row shows the image's name, format badge, size, last-modified time, and full path, with a **Download** button that streams the file to your browser.
 - Use the **Custom Path** field to look at images outside the default directory: enter a file or directory path and click **Browse** to list what's there, or **Download** to fetch a specific file path directly (pressing Enter in the field also triggers a download).
 - **Refresh** reloads the image list from the host.
5. **Empty / fail**: Check health, auth, and domain dependencies.
 6. **Success**: Live data loads; mutations complete without error toasts.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Image Builder

Purpose

Image Builder — build custom VM disk images from scratch using [mkosi](#), by picking a Linux distribution and a package list, and track builds from queued through to a finished image.

When to use it

- To produce a custom VM image (e.g. Fedora with a specific package set) instead of hand-crafting one or downloading a generic base image
- To watch a build's status live as it progresses through pending / building / completed / failed
- To see the images an mkosi build already produced, alongside their format, size, and path

How to get there

- Route / id: `/image-builder`
- Nav: **More — images, migrations & managers** → **Image Builder** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Build Image** — opens a dialog to configure a new build: an **Image Name**, a **Distribution** (Fedora, Ubuntu, Debian, CentOS, Arch Linux, or openSUSE), and a comma-separated **Packages** list (defaults to `systemd,openssh-server,iproute,vim-minimal`). Click **Build** to queue it; the page polls for updates every 5 seconds while builds are in progress.
2. **Active Builds** — shows any build currently `pending` or `building` , with its name, distribution, and state badge.
3. **Available Images** — lists finished images with name, format badge, size, and path.
4. **Build History** — shows completed and failed builds with name, distribution, status, and start time.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

ISO Images

Purpose

ISO Images — a read-only inventory of installer and driver ISO files sitting in the host's configured images directory, showing which VMs currently have each one attached.

When to use it

- Before creating a VM that boots from an OS installer ISO, to confirm the file is present on the host and copy its exact path
- To check whether a VirtIO driver ISO (`virtio-win`) is available for installing or upgrading Windows guest drivers
- To see which VMs have a given ISO attached before deleting or replacing that file on the host
- To search a large images directory by name or path

How to get there

- Route / id: `/iso-images`
- Nav: **More — images, migrations & managers** → **ISO Images** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

- **Search** ISOs by name or path.
- Each row shows the ISO's name, path, size, last-modified time, and the list of VMs that currently have it attached (**Attached VMs**); ISOs named for `virtio-win` get a small **virtio-win** badge next to the name.
- The list auto-refreshes every 30 seconds, and you can trigger an immediate refresh from the page header.
- If no ISOs are found, the empty state reminds you to place ISO files in the configured images directory on the host — this page doesn't upload or manage ISOs itself, only lists them.

5. **Empty / fail:** Check health, auth, and domain dependencies.

6. **Success:** Live data loads; mutations complete without error toasts.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Job Monitor

Purpose

Job Monitor — a live view of background jobs (disk conversions, migrations, and other pipeline work), with per-job progress, pipeline stage, and streaming logs.

When to use it

- To watch a long-running migration or conversion job progress in real time
- To find out why a job failed — the error detail and log output are right next to each other
- To check which pipeline stage a job is currently in (prepare, convert, validate, deploy)

How to get there

- Route / id: `/job-monitor`
- Nav: **More — images, migrations & managers** → **Job Monitor** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Job list** (left) — every job as a card: name (or VM name), a status badge (pending, running, completed, failed, cancelled) with icon, and a progress bar with current step. The list polls every 3 seconds, so running jobs update live.
2. **Select a job** to open its detail panel on the right: Status, Progress, Phase, and Duration tiles, plus a pipeline-stage tracker (**prepare → convert → validate → deploy**) that highlights the completed stages in green and the current stage in blue (or red if the job failed). A failed job also shows its error message in a dedicated panel.
3. **Logs** — streams the selected job's log output, polling every 2 seconds. Toggle **Follow** to auto-scroll to the newest lines, or uncheck it to scroll back through earlier output without being pulled back down.
4. Use the header **refresh** control to force an immediate reload of the job list.
5. This page is read-only monitoring — there's no way to create, edit, retry, or cancel a job from here.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Manifest Builder

Purpose

Manifest Builder — a client-side form for assembling a VM configuration manifest and exporting it as YAML, with a live preview as you type. It doesn't create a VM or call the API; it's a scratchpad for drafting config to copy elsewhere.

When to use it

- To hand-draft a VM manifest for a provisioning script, IaC pipeline, or ticket without going through the Create VM wizard
- To sketch settings the Create VM wizard doesn't expose — TPM, Secure Boot, CPU mode, console type, or `user / bridge / tap` networking — and get syntactically valid YAML for them
- To generate a quick YAML snippet to share with a teammate or paste into documentation

How to get there

- Route / id: `/manifest-builder`
- Nav: **More — images, migrations & managers** → **Manifest Builder** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Fill in fields under four collapsible sections — **VM Configuration** (expanded by default), **Network**, **Storage**, and **Advanced** — clicking a section header expands or collapses it.
2. Set VM name, vCPUs, memory, disk image path, and firmware (`uefi / bios`) under VM Configuration; network type (`user / bridge / tap`), bridge name, and MAC address under Network; disk format (`qcow2 / raw / vmdk`), disk size, and a read-only-root checkbox under Storage; and TPM, Secure Boot, console type (`serial / virtio / none`), and CPU mode (`host / max / qemu64`) under Advanced.
3. Leave any field blank or unchecked — it's simply omitted from the generated YAML, so you only need to fill in what matters for your use case.
4. Watch the **YAML Preview** pane update live as you type, grouped by section with a generated timestamp comment at the top.
5. **Copy** the YAML to your clipboard, or **Download** it as a `.yaml` file named after the VM (falls back to `vm-manifest.yaml` if no name is set).
6. **Empty / fail:** Check health, auth, and domain dependencies.
7. **Success:** Live data loads; mutations complete without error toasts.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

History

Purpose

Migration History — a read-only log of completed and failed migration jobs, with status, timing, and where the output landed.

When to use it

- To check whether a past migration succeeded or failed
- To read the error message left behind by a failed migration
- To see how long a migration took, or where its output disk was written

How to get there

- Route / id: `/migration-history`
- Nav: **More — images, migrations & managers → History** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. The page loads history from the migrations API as soon as you open it; use the header **Refresh** control to reload.
2. Table columns: **Name**, **VM**, **Status** (completed / failed / running badge, with the error message shown inline under failed rows), **Started**, **Duration**, and **Output** (output path).
3. There are no filters, search, or per-row actions here — it's a plain historical log. If nothing has run yet, you'll see "No migration history yet."

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Readiness

Purpose

Migration Readiness — pre-flight checks that verify the environment is in a good state before you start a migration, with a pass/fail summary and per-check detail.

When to use it

- Before kicking off a migration, to confirm the environment is actually ready
- To see how many errors vs. warnings are blocking readiness before digging in
- To read the specific message (and detail line) behind a failing or warning check

How to get there

- Route / id: `/migration-readiness`
- Nav: **More — images, migrations & managers → Readiness** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. The page runs readiness checks as soon as you open it; use the header **Refresh** control to re-run them.
2. A summary banner at the top reads either **Ready for Migration** (all checks passed) or **Issues Found**, with a count of errors and warnings.
3. Below it, every individual check is listed with a status icon (green check / yellow warning / red x), its name, a short message, an optional monospace detail line, and a status badge.
4. This page is read-only — it reports state but doesn't fix anything; resolve issues elsewhere, then come back and refresh.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Report

Purpose

Migration Report — a shareable summary of all migration jobs (totals by status, average duration) plus the full per-migration detail table, with copy and print actions.

When to use it

- To get an at-a-glance count of successful, failed, and running migrations
- To copy a plain-text summary of migrations for a status update or ticket
- To print or save the report as a PDF for a record or audit

How to get there

- Route / id: `/migration-report`
- Nav: **More — images, migrations & managers → Report** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. The page generates a report as soon as you open it; use the header **Refresh** control to regenerate it.
2. Once data loads, two header actions appear: **Copy Report** — copies a formatted plain-text summary (totals, average duration, and each migration's name/status/VM/duration/error) to your clipboard — and **Print** — opens the browser print dialog with a print-optimized layout.
3. Five summary tiles: **Total, Successful, Failed, Running, Avg Duration**.
4. A full table below lists every migration: Name, VM, Status badge, Duration, Output path, and Error. If nothing has run yet, you'll see "No migration data."

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Migration Templates

Purpose

Migration Templates — reusable migration configuration presets (disk format, vCPUs, memory, network, compression) that you can copy as JSON instead of re-entering the same settings for every migration.

When to use it

- To reuse a standard configuration — e.g. Production Linux, Dev/Test Linux, Windows Server — instead of retyping settings each time
- To define your own template for a migration shape you repeat often
- To grab a ready-made JSON config snippet to paste into a migration elsewhere

How to get there

- Route / id: `/migration-templates`
- Nav: **More — images, migrations & managers** → **Migration Templates** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Three **built-in** templates are always available — Production Linux (4 vCPU / 8 GB, bridged, qcow2, compressed), Dev/Test Linux (2 vCPU / 2 GB, user networking, uncompressed), and Windows Server (4 vCPU / 16 GB, bridged, qcow2, compressed, virtio). They're tagged **Built-in** and can't be deleted.
2. **New Template** opens a form — Name, Description, Format (QCOW2 / RAW / VMDK), vCPUs, Memory (MB) — **Add** saves it as a custom template.
3. Each template card shows its format, vCPU, memory, and network at a glance. The **Copy** icon copies the config as JSON to your clipboard (confirmed with a checkmark).
4. Custom templates get a **Delete** (trash) icon; built-ins don't.
5. Custom templates are saved to your browser's local storage only — they aren't a server-side resource, so they won't follow you to another browser or survive clearing site data.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Network Topology

Purpose

Network Topology — a live map of which VMs are attached to which virtual networks or host bridges, alongside the host's own bridges and physical NICs, auto-refreshing every 15 seconds.

When to use it

- To see which VMs share a network or bridge, or find VMs that aren't attached to anything
- To check a host bridge's or physical NIC's operational state and addresses
- To look up a VM's interface type, MAC address, and which network it plugs into without opening each VM individually

How to get there

- Route / id: `/network-topology`
- Nav: **More — images, migrations & managers** → **Network Topology** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. The page loads VM interfaces, virtual networks, host bridges, and links on open, then refreshes silently every 15 seconds; the header **Refresh** forces a full reload.
2. A summary line shows counts of virtual networks, host bridges, and VMs.
3. **Host interfaces** — cards for each host bridge and physical NIC, showing whether it's a bridge or interface, its operational state, and its addresses.
4. **VM → network connections** — a flat table of every VM interface: VM name and state, the network/bridge it points to, interface type, and MAC address.
5. **VM attachments** — one card per network/bridge listing the VMs attached to it (with per-interface type/source/model/MAC); VMs with no attached interface are grouped under "Unattached VMs."
6. This page is read-only — to change what a VM is attached to, use the [Create VM](#) wizard or that VM's own Network tab.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Pipeline

Purpose

Pipeline Monitor — a live, auto-refreshing view of in-progress migration/conversion jobs, showing each job's percent complete and which of five stages it's currently in.

When to use it

- To watch a migration or conversion in progress and see exactly which stage it's at
- To catch a failure as it happens instead of waiting for the history page to update
- To find a job's duration or output path as soon as it's available

How to get there

- Route / id: `/pipeline`
- Nav: **More — images, migrations & managers → Pipeline** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. The page loads active jobs on open, then polls silently every 3 seconds; the header **Refresh** forces a manual reload.
2. Each job card shows the VM name, job ID, source, and a status badge (pending / running / completed / failed).
3. A progress bar shows percent complete.
4. A stage tracker walks through **inspect → prepare → convert → validate → deploy**: completed stages are green, the current stage pulses blue (red if the job failed), and remaining stages are grey.
5. Duration and output path appear once available; a failed job shows its error message in a red panel below.
6. This is a read-only monitor — there's no start, cancel, or retry action here.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Snapshot Mgr

Purpose

Snapshot Manager — create, revert to, and delete disk-state snapshots for a selected VM.

When to use it

- Before a risky change to a VM, to capture its disk state so you can roll back
- To revert a VM to an earlier snapshot after something went wrong
- To clean up old snapshots you no longer need, or check a snapshot's creation time and parent

How to get there

- Route / id: `/snapshot-manager`
- Nav: **More — images, migrations & managers** → **Snapshot Mgr** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Select VM** — pick a VM from the dropdown to load its snapshots; the list auto-refreshes every 15 seconds while a VM is selected, or use **Refresh** to reload on demand.
2. **Create Snapshot** — enter a name and click **Create** to snapshot the selected VM's current disk state; a success banner confirms it.
3. The snapshots table lists Name, Created timestamp, State, and Parent, with two actions per row:
 - **Revert** (circular arrow) — reverts the VM to that snapshot, behind a confirmation dialog
 - **Delete** (trash) — deletes the snapshot, behind a confirmation dialog
4. Empty states: no VM selected prompts "Select a VM"; a VM with no snapshots shows "No snapshots."

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Storage Mgr

Purpose

Storage Manager — browse storage pools with their capacity and usage, and drill into a pool to see its volumes.

When to use it

- To check how full a storage pool is before provisioning new disks on it
- To see what volumes live in a given pool, and their format and size
- To find a volume's on-disk path

How to get there

- Route / id: `/storage-manager`
- Nav: **More — images, migrations & managers** → **Storage Mgr** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Pools load on open; the header **Refresh** reloads them.
2. Each pool is a card showing its name, a type badge (dir / logical / netfs / disk / iscsi / rbd / zfs), a state badge, a usage bar (blue, turning amber above 70% and red above 90%), and used / capacity / available figures.
3. Click a pool card to select it and load its volumes.
4. The volumes table for the selected pool lists Name, Format, Capacity, Allocation, and Path.
5. This page is read-only — there's no create, resize, or delete action for pools or volumes here.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Upload Disk

Purpose

Upload Disk Image — drag-and-drop (or browse) upload of a VM disk image file to the server, with live progress and an in-session upload history.

When to use it

- To get a disk image onto the host so it can be used to create a VM
- To upload a disk exported or converted elsewhere for import
- To check what you've uploaded recently in this session

How to get there

- Route / id: `/upload-disk`
- Nav: **More — images, migrations & managers** → **Upload Disk** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. The drop zone accepts drag-and-drop or click-to-browse. Only `.qcow2`, `.vmdk`, `.vhd`, `.vhdx`, `.raw`, `.img`, and `.ova` are accepted — anything else is rejected with an inline error.
2. Once a file is chosen, set the **Destination Directory** (defaults to `/var/lib/libvirt/images`) and click **Upload**.
3. The upload runs with a live progress bar — percent complete, bytes transferred/total, and transfer speed — and a **Cancel** button to abort mid-upload.
4. On success, a confirmation banner shows the saved path, format, and size, and the file is added to the **Upload History** list below (name, format, size, time) — this list only lasts for the current session and resets on page reload.
5. On failure — rejected format, network error, or cancellation — an error banner explains what happened.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Autoscale

Purpose

Autoscale — define per-VM policies that automatically grow or shrink a VM's vCPUs and memory within set bounds based on load, and review the history of scaling actions that were triggered.

When to use it

- To let a VM's resources flex automatically instead of manually resizing it under load
- To cap how far a VM is allowed to scale (min/max vCPUs, min/max memory) so autoscaling can't run away
- To check what scaling actions actually fired and when, via the recent scale events log

How to get there

- Route / id: `/autoscale`
- Nav: **Operations** → **Autoscale** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review the policy table — one row per VM, showing its vCPU range, memory range, CPU scale-up/scale-down thresholds, and cooldown period (seconds between scaling actions).
2. **Create Policy** opens a form to pick a VM (only VMs without an existing policy are offered) and set min/max vCPUs, min/max memory (MB), CPU scale-up and scale-down thresholds (%), and the cooldown in seconds.
3. **Delete** a policy from its row (confirmation required) to stop autoscaling that VM.
4. Check **Recent scale events** — a scrolling log of the last 20 scaling actions (VM, action, resource, timestamp) so you can see what autoscale actually did.
5. On a read-only account, the create/delete controls are hidden and a read-only notice is shown instead.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Backups

Purpose

Backups & Restore — create full or incremental VM backups, track running backup/restore jobs, and restore a VM from a completed backup, either in place or as a new VM.

When to use it

- To take a one-off backup of a VM before a risky change, or as part of a regular backup routine
- To check on a backup or restore job that's currently running
- To roll a VM back to a previous backup, or clone it into a new VM from that backup

How to get there

- Route / id: `/app/backups`
- Nav: **Operations** → **Backups** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review the summary tiles: total backups, total size, a breakdown by type (full vs. incremental), and the timestamp of the newest backup.
2. Watch **Active Jobs** — running or queued backup/restore jobs with a live progress bar, refreshed automatically every 5 seconds.
3. **Create Backup** opens a dialog to pick a VM and choose **Full** or **Incremental**; the job is created compressed with a 30-day retention by default.
4. Search the backup table by VM name, type, or status, and review each backup's size, created time, expiry (or "Never"), and status (completed / in progress / failed).
5. **Restore** (enabled only once a backup is `completed`) opens a dialog where you either restore over the original VM, or check **Restore to new VM** and give it a name to clone the backup into a fresh VM instead. Restoring always brings back config and disks; VM runtime state is not restored.
6. **Delete** a backup permanently, after a confirmation dialog warning the action can't be undone.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Bulk Operations

Purpose

Bulk Operations — select any number of VMs and start, stop, restart, or snapshot them together, with a per-VM progress log for the batch.

When to use it

- To restart or stop a group of VMs at once (e.g. before a maintenance window) instead of one at a time
- To take a quick snapshot of several VMs together before a risky change
- To see which VMs in a batch action succeeded and which failed, with the specific error for each

How to get there

- Route / id: `/app/bulk-operations`
- Nav: **Operations** → **Bulk Operations** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Filter the VM list by name, then click rows to select VMs (or use **Select All** / **Deselect All** for whatever the current filter shows).
2. Once at least one VM is selected, an action bar appears with **Start**, **Stop**, **Restart**, and **Snapshot** — snapshot creates a timestamped snapshot (`bulk-snap-<timestamp>`) on each selected VM.
3. Each action runs sequentially across the selected VMs (no confirmation dialog — the action fires as soon as you click it), with **Batch Progress** showing a live per-VM status: pending → running → done/error, plus the specific error message if one VM fails while others succeed.
4. **Clear** deselects everything, and **Clear results** dismisses the batch progress log once a run finishes.
5. The VM table also shows each VM's state, CPU count, and memory as a quick reference while selecting.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Content Library

Purpose

Content Library — a catalog of reusable provisioning building blocks: libraries of templates/ISOs/OVFs/scripts, guest customization specs (per-OS hostname/domain/DNS settings), and host compliance profiles.

When to use it

- To organize VM templates, ISOs, OVF packages, and scripts into named libraries instead of scattering them across storage paths
- To define a reusable guest customization spec (hostname prefix, domain, DNS servers) for Linux or Windows guests
- To check host compliance profile status — how many hosts are compliant vs. non-compliant against a profile

How to get there

- Route / id: `/app/content-library`
- Nav: **Operations** → **Content Library** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Summary tiles at the top show total libraries, items, guest customization specs, and host profiles. Four tabs underneath:

1. **Libraries** — cards showing each library's status, item count, and total size. **Create Library** names it, sets an optional description, chooses **Local** or **Subscribed** type, and requires a storage path. **Browse** jumps to the Item Browser tab filtered to that library; the trash icon deletes a library and all its items (confirmation required).
2. **Item Browser** — a table of every item across libraries (template, ISO, OVF, script, or file), with type, version, size, and last-updated date. Filter to one library via the dropdown, or view all. Delete an item from its row (confirmation required).
3. **Guest Customization** — a table of customization specs (OS type, hostname prefix, domain, DNS servers). **Create Spec** sets the name, Linux/Windows OS type, hostname prefix, domain, and a comma-separated DNS server list. Delete a spec from its row (confirmation required).
4. **Host Profiles** — a table of profiles with compliant/non-compliant host counts and status. **Create Profile** sets a name and optional description. Delete a profile from its row (confirmation required).

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

DRS

Purpose

Distributed Resource Scheduler (DRS) — balances VM placement across the hosts in a cluster, surfaces migration recommendations, enforces affinity/anti-affinity rules, and can test where a new VM would land before you create it.

When to use it

- To see how evenly CPU and memory load is spread across your cluster's hosts
- To review (and approve or reject) migrations DRS suggests to rebalance load
- To keep certain VMs together (affinity) or apart (anti-affinity) — e.g. two replicas of the same service on different hosts
- To test where a hypothetical VM (given its vCPU/memory needs) would be placed before actually creating it

How to get there

- Route / id: `/app/drs`
- Nav: **Operations** → **DRS** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

DRS configuration (top of page) — toggle DRS **On/Off**, set the **Automation Level** (Manual, Semi-Auto, or Fully Auto), adjust the **Migration Threshold** (1–5, how aggressively it rebalances), and see the configured check interval. Changes apply immediately.

Four tabs below that:

1. **Balance** — the overall cluster balance score plus CPU/memory imbalance percentages, and a per-host breakdown with live CPU and memory usage bars (color-coded amber/red as usage climbs).
2. **Recommendations** — a table of migrations DRS suggests (VM, source host, target host, reason, priority, estimated benefit). Pending recommendations can be **approved** (checkmark) or **rejected** (X) directly from the row.
3. **Rules** — affinity/anti-affinity rules, each showing type, VM count, whether it's mandatory, and whether it's enabled. **Create Rule** sets a name, chooses **Affinity** (keep together) or **Anti-Affinity** (keep apart), lists VM IDs (comma-separated), and an optional mandatory flag. Delete a rule from its row (confirmation required).
4. **Placement** — a calculator: enter a hypothetical VM's CPU (MHz) and memory (MB), click **Test Placement**, and see the recommended host, its score and reasoning, projected CPU/memory usage after placement, and scored alternative hosts.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Fault Tolerance

Purpose

Fault Tolerance (FT) — protect individual VMs with a live secondary replica on another host, so a host failure fails the VM over instead of taking it down, and monitor replication health.

When to use it

- To keep a critical VM running through a host failure by giving it a hot secondary on another host
- To check whether a VM is actually FT-compatible before enabling it, and fix any blocking issues first
- To test a failover safely, or trigger a real one when you need to move a VM off its current host
- To watch replication health — log bandwidth, checkpoint latency, secondary host load, uptime — for protected VMs

How to get there

- Route / id: `/app/fault-tolerance`
- Nav: **Operations** → **Fault Tolerance** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Summary tiles show protected VM count, how many are running, how many need attention, and total failover events. **Enable FT** opens a form to name the VM, set its primary host ID, and optionally a secondary host ID (leave blank to auto-select). Four tabs:

1. **Protected VMs** — table of FT-enabled VMs with their secondary host, status, and bandwidth limit. Per row: **Test** (a non-disruptive failover test), the play icon to **trigger a real failover** (confirmation required — moves the VM to its secondary host), and the trash icon to **disable FT** (confirmation required).
2. **Compatibility Check** — enter a VM ID and check whether it's FT-compatible; results show a pass/fail badge, any blocking or warning issues (each with a suggested fix), and recommended secondary hosts with a compatibility score.
3. **Events** — a timeline of failover events (test and real), each showing status, failover type, source → target host, downtime in ms, timestamp, and the error message if it failed.
4. **Metrics** — per protected VM: log bandwidth usage, checkpoint latency, secondary host's CPU/memory usage, failover count, and uptime percentage.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Lifecycle

Purpose

Lifecycle Manager — define patch/upgrade baselines, scan hosts for compliance against them, remediate non-compliant hosts, and track rolling updates across a host fleet.

When to use it

- To define a baseline (patch, upgrade, or extension) with a severity level, and see which hosts already meet it
- To scan hosts for compliance against a baseline and find out which are missing patches
- To watch a remediation task apply patches to a host, or a rolling update roll out across a fleet host by host

How to get there

- Route / id: `/app/lifecycle`
- Nav: **Operations** → **Lifecycle** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Summary tiles show total baselines, non-compliant hosts, active remediation tasks, and currently-running rolling updates. Four tabs:

1. **Baselines** — table of baselines (type, severity, release date, host count, compliant count, and a compliance progress bar). **Create Baseline** sets a name, optional description, type (Patch/Upgrade/Extension), and severity (Critical/Important/Moderate/Low). Per row: the play icon **runs a compliance scan** against that baseline, and the trash icon **deletes it** (confirmation required).
2. **Compliance Scans** — results per host: which baseline it was scanned against, status (compliant / non-compliant / incompatible / etc.), missing patch count, and when it was last scanned.
3. **Remediation** — active and past remediation tasks per host: status, a progress bar, patches applied vs. total, and any error message.
4. **Rolling Updates** — each update plan shown as a card with status, hosts completed vs. total, parallelism (how many hosts update at once), the host currently being updated, a progress bar, start/completion timestamps, and a failed-host count if any.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Migration Wizard

Purpose

Migration Wizard — a three-step wizard (Source → Configure → Review) for converting an existing disk image (local file or remote host) into a new Zylvor Fabric VM.

When to use it

- To bring in a VM from elsewhere — a disk image sitting on the local filesystem, or one reachable over SSH on a remote host
- To convert a disk image to a different format (QCOW2, RAW, or VMDK) as part of importing it
- To size the resulting VM's vCPUs and memory and decide whether it should auto-start once migration finishes

How to get there

- Route / id: `/migration-wizard`
- Nav: **Operations** → **Migration Wizard** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Source — choose **Local File** (a disk image path, e.g. `/path/to/disk.vmdk`) or **Remote Host** (an `user@hostname:/path` target reachable over SSH).

2. Configure — set the new VM's name, target disk format (QCOW2, RAW, or VMDK), vCPUs (1–64), memory (MB, in 256 MB steps), the output directory for the converted image (defaults to `/var/lib/zyvor-fabricd/images`), and whether to auto-start the VM once migration completes.

3. Review & Submit — shows a summary of everything chosen, then **Submit Migration** posts the job. Success or failure is shown inline on this step (with hints on failure); you can't resubmit once it succeeds. Use **Back** at any point to revisit an earlier step before submitting.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Migrations

Purpose

VM Migrations — move a VM from its current host to a different target host, and track the migration from start to finish. The list auto-refreshes every 5 seconds so in-flight migrations update live.

When to use it

- To move a VM off a host for maintenance or to rebalance load
- To watch the progress of a migration currently in flight (percent complete, bytes transferred)
- To cancel a migration that's stuck or no longer needed
- To review past migrations — which VM, which target host, which type, and how they ended

How to get there

- Route / id: `/app/migrations`
- Nav: **Operations** → **Migrations** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Start Migration** — opens a dialog where you pick a VM from the dropdown (populated from your VM list), enter a target host (hostname or IP), and choose a migration type:
 - **Offline** — stop the VM, copy its data, then start it on the target
 - **Live** — migrate with minimal downtime
 - **Storage** — migrate storage volumes only
2. **Active Migrations** — each in-progress migration shows as a card with a live progress bar (percent complete), bytes transferred, its state badge (pending, precheck, syncing, switching), and a **Cancel** button. Any error from a failed migration is shown inline on the card.
3. **Migration History** — a table of completed, failed, and cancelled migrations showing VM, target host, migration type, final status, and start time.
4. **Refresh** — manually reload the list; the page also polls automatically every 5 seconds.
5. If there are no migrations at all, the page shows an empty state with a shortcut into the Start Migration dialog.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Quotas

Purpose

Resource Quotas — cap CPU, memory, disk, and VM-count usage, applied either globally or to VMs matching specific tags, so a team or workload can't consume unlimited host resources.

When to use it

- To limit how many CPUs, how much memory/disk, or how many VMs a team can create
- To scope a limit to a subset of VMs by tag, rather than the whole host
- To check whether a quota is currently exceeded and which resource pushed it over
- To temporarily disable a quota without deleting its configuration

How to get there

- Route / id: `/app/quotas`
- Nav: **Operations** → **Quotas** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Create Quota** — opens a form for the quota name, four numeric limits (max CPUs, max memory in MB, max disk in GB, max VMs), and optional tags to scope it to tagged VMs only (leave tags empty to apply globally). A checkbox lets you enable the quota immediately on creation.
2. Each quota renders as a card with **Enabled/Disabled** and **Exceeded** badges, and four usage bars (CPUs, memory, disk, VMs) each showing used/limit and a percentage, colored green under 75%, yellow at 75–89%, and red at 90%+.
3. If a quota is exceeded, the card shows a warning listing which resources are over limit and notes that new VMs matching this quota can't be created until usage drops.
4. Per-quota actions: **Enable/Disable** (toggle without deleting), **Edit** (opens the same form pre-filled, showing current usage and warning if a new limit would go below what's already in use), and **Delete** (confirmation dialog, cannot be undone).
5. If no quotas exist yet, the page shows an empty state with a shortcut to create the first one.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Replication

Purpose

Replication — register remote replication sites, configure per-VM replication to them with a target RPO (recovery point objective), and monitor sync health and RPO compliance across your fleet.

When to use it

- To register a secondary (or bidirectional) site that VMs can replicate to
- To start replicating a VM to another site with a target RPO in minutes
- To pause or resume replication for a VM without tearing down the configuration
- To check overall replication health, or find which VMs are currently violating their RPO target

How to get there

- Route / id: `/app/replication`
- Nav: **Operations** → **Replication** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Dashboard tab** — summary tiles for active replications, RPO violation count, average RPO across all replications, and paused/error counts, plus a per-site health list (healthy/degraded/unhealthy) with each site's replication count.
2. **Sites tab** — table of registered replication sites (name, type, endpoint, status, replication count, last sync). **Add Site** registers a new site with a name, endpoint URL, and type (Primary, Recovery/secondary, or Bidirectional). Each site can be removed (confirmation dialog).
3. **Replications tab** — table of per-VM replication configs showing target RPO, status, live sync progress bar, last/next sync time. **Configure Replication** sets up a new one: VM ID, source site, target site, and RPO in minutes. Active replications can be **paused**; paused ones can be **resumed**.
4. **RPO Violations tab** — table of replications currently missing their target RPO, showing target vs. current RPO, compliance status, bandwidth usage, sync count, and failure count. Shows "All replications are compliant" when there are none.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Schedules

Purpose

VM Schedules — automate a recurring lifecycle action (start, stop, restart, or snapshot) for a single VM, on a one-time, daily, or weekly schedule.

When to use it

- To automatically stop dev/test VMs overnight or on weekends to save resources
- To take a recurring snapshot of a VM on a schedule
- To run a one-off action at a specific future time without staying logged in
- To check whether a scheduled action actually ran, and whether it succeeded

How to get there

- Route / id: `/app/schedules`
- Nav: **Operations** → **Schedules** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Create Schedule** — pick a VM, an action (Start VM, Stop VM, Restart VM, Create Snapshot), a schedule type, and a time (24-hour UTC):
 - **Once** — runs a single time
 - **Daily** — runs every day at the given time
 - **Weekly** — pick one or more days of the week to run on A checkbox enables the schedule immediately on creation.
2. Search schedules by name, VM, or action using the search box above the list.
3. Each schedule shows as a card with its enabled/disabled state, action badge, target VM, a human-readable description of its cadence (e.g. "Weekly on Mon, Wed at 09:00"), and next/last run times (relative).
4. Per-schedule actions: **Run Now** (executes immediately, confirmation required, disabled when the schedule itself is disabled), **Enable/Disable** (toggle without deleting), **Edit** (change name, action, cadence, or time), and **Delete** (confirmation, cannot be undone).
5. **History** button switches the page to an execution history table (latest 20 runs) showing schedule, VM, action, when it ran, and success/failure status with the error message if it failed.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Site Recovery

Purpose

Site Recovery — define disaster recovery plans that group VMs by source and target site, then execute those plans as a test failover, a planned migration, or a full disaster recovery, and track how each execution unfolds.

When to use it

- To set up which VMs fail over together, and to which target site, ahead of an actual incident
- To run a non-destructive test failover to validate a plan works before you need it for real
- To execute an actual disaster recovery when a site goes down
- To check fleet-wide DR posture — how many VMs are protected, average RTO/RPO, and which plans have been tested

How to get there

- Route / id: `/app/site-recovery`
- Nav: **Operations** → **Site Recovery** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **DR Dashboard tab** — summary tiles for protected vs. unprotected VMs, average RTO, average RPO, and plans tested vs. total plans; plus a per-site status list and the 5 most recent executions with their status.
2. **Recovery Plans tab** — table of plans (name/description, VM group count and total VMs, status, last tested date, last executed date). **Create Plan** defines a plan name, description, source site ID, target site ID, a VM group name, and a comma-separated list of VM IDs in that group.
3. Each plan can be **executed** — opens a modal with three execution types, each with different risk:
 - **Test Failover** — non-destructive, runs in an isolated network
 - **Planned Migration** — graceful, zero data loss
 - **Disaster Recovery** — emergency failover; some data loss possible; confirmation dialog warns this fails over all VMs to the target site Plans can also be **deleted** (confirmation dialog).
4. **Execution History tab** — each past/in-progress execution shows plan name, execution type, status, VMs recovered vs. total, actual RTO once known, a live progress bar with the current step, a chip per step showing its individual status, and start/completion timestamps.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Snapshots

Purpose

VM Snapshots — create point-in-time snapshots of a specific VM's disk (or disk + state), and revert or delete them. Unlike most Operations pages, this one is scoped to one VM at a time, entered by name.

When to use it

- To capture a VM's disk state before a risky change, then roll back if needed
- To browse the snapshot history for a specific VM
- To revert a VM to an earlier snapshot
- To clean up old snapshots you no longer need

How to get there

- Route / id: `/app/snapshots`
- Nav: **Operations** → **Snapshots** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Type a VM name into the **VM Name** field and click **Load Snapshots** (or wait — snapshots load automatically once a name is entered) to see that VM's snapshot list. Nothing loads until a VM name is provided.
2. **Create** — opens a dialog for a snapshot name, an optional description, and a type: **Disk Only** or **Full (Disk + State)**.
3. The snapshot table lists each snapshot's name, type, description, and relative creation time.
4. Per-snapshot actions: **Revert to snapshot** (confirmation dialog warns the VM must be stopped first) and **Delete snapshot** (confirmation dialog).
5. If the VM has no snapshots yet, the table shows "No snapshots found for this VM."

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Templates

Purpose

VM Templates — reusable VM configurations (CPU/memory/disk, tags) that you save from an existing VM and use to stamp out new VMs quickly, instead of configuring resources from scratch each time.

When to use it

- To create a new VM with the same resource configuration as one you've already tuned
- To standardize resource sizing across a team (e.g. a "small-dev" or "large-prod" template)
- To browse the templates already saved and their specs before deciding which to use

How to get there

- Route / id: `/app/templates`
- Nav: **Operations** → **Templates** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Templates aren't created directly on this page — click **Create from VM** to jump to the [Virtual Machines](#) page, where a template is saved from an existing VM's configuration.
2. Each saved template shows as a card with its name, description, CPUs, memory (MB), disk size (GB), tags, and creation date.
3. **Create VM** on a template card opens a dialog asking only for a new VM name, then deploys a new VM using that template's saved resource configuration.
4. **Delete** removes a template (confirmation dialog, cannot be undone) — this does not affect VMs already created from it.
5. If no templates exist yet, the page shows an empty state pointing you to the VMs page to create one.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Access Control

Purpose

Access Control — manage the user accounts that can sign in to Zylvor Fabric: create accounts, assign a role (admin, operator, or viewer), and enable/disable or delete them.

When to use it

- To create a login for a new team member and decide up front what they're allowed to do
- To temporarily lock a user out without deleting their account
- To see at a glance who has admin access, or when someone last logged in
- To remove an account that's no longer needed

How to get there

- Route / id: `/app/access-control`
- Nav: **Security** → **Access Control** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review the four stat tiles — Total Users, Admins, Operators, Viewers — for a quick read on who has access.
2. Click **Add User** to open the inline form: enter a username (3-32 characters — letters, numbers, hyphens, underscores), a password (8+ characters), and pick a role by clicking **Admin**, **Operator**, or **Viewer**. Bad input is caught client-side before it's submitted.
3. Review the user table: avatar/username, role badge, an **Active/Disabled** toggle, created date, and last login (or "Never" if they haven't signed in).
4. Flip a user's **Active/Disabled** toggle directly in the table to lock them out or restore access without deleting the account.
5. Delete a user with the trash icon — a confirmation dialog ("Delete user "..."?") must be accepted first.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Certificates

Purpose

Certificates & Security — a PKI console for Zyvor Fabric: certificate authorities, issued certificates, the CSR approval queue, host TPM/boot attestation, and VM security baselines, rolled up into one health dashboard.

When to use it

- To see which certificates are expiring soon and renew them before they lapse
- To stand up a new certificate authority (root, intermediate, or external) for issuing certs
- To approve certificate signing requests waiting in the queue
- To check whether hosts have a TPM present and are passing boot/secure-boot attestation
- To track fleet compliance against a VM security baseline

How to get there

- Route / id: `/app/certificates`
- Nav: **Security** → **Certificates** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Six tabs: **Dashboard**, **CAs**, **Certificates**, **Requests**, **Attestation**, **Security Baselines**.

1. **Dashboard** — total certificates, active/expiring-soon/expired counts, CA count, pending requests, and overall compliance %, plus a list of certificates expiring within 30 days with days remaining.
2. **CAs** — table of certificate authorities (name, type, subject, valid-until date colored by days remaining, certs issued, status). **Create CA** opens a form: name, type (Root / Intermediate / External), and subject (e.g. `CN=My CA, O=My Org`).
3. **Certificates** — table of issued certs (subject, type, serial, expiry colored by days remaining, host/service, status). Active certificates have a **Revoke** action, gated by a confirmation dialog.
4. **Requests** — the CSR queue (subject, requestor, type, key size, submitted time, status). Pending requests can be approved with a single click; there's no reject action in this view.
5. **Attestation** — per-host TPM status: TPM present/absent, TPM version, attestation status, boot integrity verified/failed, secure boot enabled/disabled, and last check time.
6. **Security Baselines** — table of baselines (name/description, VM count, compliant count, number of checks, compliance % bar, last scan). **Create Baseline** opens a form for name and description; a default check is attached automatically.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Compliance

Purpose

Compliance Dashboard — a security and configuration compliance scorecard: an overall score, pass/warning/fail counts by category, and remediation guidance for anything that isn't passing.

When to use it

- To get a single number for how compliant your deployment currently is
- To find exactly which checks are failing or warning, and how to fix each one
- To trigger a fresh compliance scan on demand instead of waiting for the next one
- To narrow the check list down to one category, e.g. only network or only auth checks

How to get there

- Route / id: `/app/compliance`
- Nav: **Security → Compliance** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. View the compliance score (0-100, color-coded green/amber/red) alongside Passed, Warnings, Failed, and Total Checks tiles.
2. Click **Run Scan** to trigger a fresh compliance scan — the button reads "Scanning..." while it runs, and the dashboard refreshes with the new results when it completes.
3. Filter checks by category using the pill buttons (**all**, plus each category returned by the last scan).
4. Review each check's name, status (pass/warning/fail), and description; anything not passing shows a "Fix:" remediation note inline.
5. See when the data was last collected via the "Last scan" timestamp at the bottom of the page.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Encryption

Purpose

Encryption — manage VM disk and vMotion encryption: register key management providers (KMIP, HashiCorp Vault Transit, or local software keys), define reusable encryption policies, and see which VMs are encrypted under which policy.

When to use it

- To connect an external KMS so Zyvor Fabric manages encryption keys outside the host
- To define a policy (algorithm, whether vMotion traffic is encrypted, key rotation schedule) and see who's using it
- To check which VMs are currently encrypted and under which policy
- To rotate a VM's encryption key on demand, e.g. after a suspected key compromise

How to get there

- Route / id: `/app/encryption`
- Nav: **Security** → **Encryption** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

Summary tiles up top (Key Providers, Policies, Encrypted VMs, Connected Providers), then three tabs:

1. **Key Providers** — table of registered providers (name, type, endpoint, status). **Add Provider** opens a form: name, type (**KMIP**, **Local** software-based, or **HashiCorp Vault Transit**), and endpoint URL. Remove a provider with the trash icon, gated by a confirmation dialog.
2. **Policies** — table of encryption policies (name/description, key provider, algorithm, whether vMotion traffic is encrypted, auto-rotate interval). **Create Policy** opens a form: name, description, key provider (dropdown of registered providers), algorithm (**AES-256-XTS**, **AES-256-CBC**, or **ChaCha20-Poly1305**), an "Encrypt vMotion traffic" toggle, and an "Auto-rotate keys" toggle that reveals a rotation-interval-in-days field when enabled.
3. **Encrypted VMs** — table of VMs (name, encrypted yes/no, policy applied, algorithm, last key rotation date). Encrypted VMs get a **Rotate Key** action, gated by a confirmation dialog, to force an immediate key rotation.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Plugins

Purpose

Plugin Manager — enable, disable, and review the server extensions installed on Zyvor Fabric (storage, network, security, monitoring, and backup plugin types).

When to use it

- To see which plugins are installed and whether each is running, stopped, or erroring
- To turn a plugin on or off without restarting the whole service
- To check a plugin's version and author before relying on it

How to get there

- Route / id: `/plugins`
- Nav: **Security** → **Plugins** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Review the four stat tiles: Total Plugins, Running, Errors, and Types (distinct plugin categories installed).
2. Browse the plugin cards — each shows name, version, a type badge (storage/network/security/monitoring/backup), status (running/stopped/error), description, and author when provided.
3. Click **Enable/Disable** on a card to toggle it; the button shows a spinner while the request is in flight and the card's status updates immediately after.

This page is enable/disable only — there's no install or configure-plugin flow here.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Security Dashboard

Purpose

Security — a real-time security posture and threat-monitoring view: an overall risk score, active security alerts, recent failed login attempts, and listening network ports on the host. Data auto-refreshes every 5 seconds. This is distinct from [Compliance](#) (configuration checks) and [Certificates](#) (PKI health) — this page is about live threats and activity, not point-in-time audits.

When to use it

- To get a fast read on overall risk from the single risk-score gauge
- To see currently active security alerts by severity (critical/warning/info)
- To spot a brute-force or credential-stuffing attempt via the failed-logins table
- To audit which ports are open and listening on the host, and which process owns each one

How to get there

- Route / id: `/security-dashboard`
- Nav: **Security** → **Security Dashboard** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. View the risk score gauge (0-100, color-coded) alongside Critical Alerts, Warnings, and Failed Logins count tiles.
2. Review the Security Alerts list — severity badge, message, source, and timestamp per alert; shows "No active alerts" when clear.
3. Review the Failed Logins table (time, user, source) — only appears when there are failed attempts to report.
4. Review the Listening Ports table (port, protocol, process, PID) — only appears when the host has ports open.
5. Data refreshes automatically every 5 seconds; a manual refresh is also available from the page header.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Cost Estimator

Purpose

Storage Cost Estimator — a what-if calculator that projects cloud storage cost (AWS S3, Azure Blob, GCS) for your VM fleet and compares it against an on-premises baseline. It's a planning tool: figures come from the inputs you set, not from your actual billing or usage.

When to use it

- To budget or justify a move to cloud storage before you commit to it
- To see how fleet size, average VM disk size, or snapshot retention changes projected storage spend
- To put a number on "how much would we save vs. on-prem" for a proposal or ticket

How to get there

- Route / id: `/cost-estimator`
- Nav: **Tools** → **Cost Estimator** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Configuration** — set Number of VMs (1–1000), Average VM Size (10 GB–2 TB), and Storage Duration (1–36 months) with sliders. Toggle **Include snapshots** to add 50% to the total storage figure.
2. Click **Calculate Costs**. The page first tries the backend (`POST /api/cost/estimate`); if that call fails or returns no estimates, it falls back to a client-side calculation using fixed per-GB rates (AWS S3 \$0.023, Azure Blob \$0.018, GCS \$0.020, on-prem \$0.10).
3. Results show three provider cards (Monthly / Annual / Total for the chosen duration), with the cheapest option badged.
4. A savings panel compares the cheapest cloud option against the on-prem estimate, with a percentage saved.
5. A horizontal bar chart compares monthly cost across all three cloud providers plus on-prem.
6. **Copy Estimate to Clipboard** copies a plain-text summary of your inputs and the results — handy for pasting into a ticket or email.
7. **Empty / fail**: Check health, auth, and domain dependencies.
8. **Success**: Live data loads; mutations complete without error toasts.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Notification Center

Purpose

Notification Center — a live, session-only tray of VM events and system alerts, polled from the server every 10 seconds. It's separate from **Monitoring → Notifications**, which manages persistent delivery channels, rules, and history; this page just surfaces what's firing right now and forgets everything when you reload.

When to use it

- To catch alerts and warnings as they happen without watching the Monitoring dashboard
- To scan a short-lived list of recent events during troubleshooting
- Not for setting up where alerts get delivered (email/Slack/webhook) — use [Notifications](#) for that

How to get there

- Route / id: `/notification-center`
- Nav: **Tools → Notification Center** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. The page polls `GET /api/system/alerts` every 10 seconds; each newly seen alert is added to the feed, tagged **Alert** or **System Warning** based on its severity. It also subscribes to the live `/api/events/stream` feed for **VM Started** / **VM Stopped** events, so all four categories populate from real activity.
2. Category chips at the top show a live count per type.
3. An unread count appears when there are unread notifications; clicking a notification marks it read (dims it).
4. Click the **x** on a notification to dismiss it individually, or **Clear All** to empty the whole feed.
5. If polling fails, an amber banner reports the error; polling keeps retrying every 10 seconds in the background.
6. **Empty / fail:** Check health, auth, and domain dependencies.
7. **Success:** Live data loads; mutations complete without error toasts.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

API Playground

Purpose

API Playground — try Fabric APIs interactively.

When to use it

- Open this surface when the job matches the purpose above
- Start from the product home / dashboard if you are unsure where to begin
- Confirm auth and that required backends/operators are reachable if data looks empty

How to get there

- Route / id: `/app/playground`
- Nav: **Tools** → **API Playground** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Open `/app/playground` and wait for live data from Zyvor Fabric.
2. Use filters and search when the page provides them.
3. Drill into a row or card for detail, then jump to related surfaces.
4. For mutating actions: review impact, role gates, and confirmation dialogs first.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

VM Compare

Purpose

VM Comparison — a side-by-side diff of two VMs' configurations, run on demand against the live VM list.

When to use it

- To see what differs between two VMs before assuming they're equivalent (useful when troubleshooting "why does this one behave differently")
- To confirm a newly cloned or templated VM actually matches its source
- To audit configuration drift between two VMs that are supposed to be identical

How to get there

- Route / id: `/vm-compare`
- Nav: **Tools** → **VM Compare** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Pick a **Source VM** and **Target VM** from two dropdowns populated from your VM list (the target list excludes whichever VM is already chosen as source).
2. Click **Compare** (disabled until both are selected) — calls `GET /api/vms/compare?source=...&target=...`.
3. Results appear as a table: one row per compared field, with Source value, Target value, and a Match column (Yes in green, No in amber) so mismatches jump out.
4. Use the refresh control in the header to re-fetch the VM list if it's out of date.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

VM Health Check

Purpose

VM Health Check — runs a set of health verification checks against a single VM, on demand, and reports pass/warning/fail per check plus an overall status.

When to use it

- Before or after a change, to confirm a VM is still in good health
- To triage a VM you suspect is unhealthy, with a per-check breakdown instead of just up/down
- Before promoting a VM to production or handing it off

How to get there

- Route / id: `/vm-healthcheck`
- Nav: **Tools** → **VM Health Check** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. Select a VM from the dropdown, populated from your VM list.
2. Click **Run Health Check** — calls `GET /api/vms/{name}/healthcheck`.
3. An overall status banner reports **Healthy** or **Issues Found**, with a "X of Y checks passed" summary.
4. Each individual check is listed with a status icon (pass / warning / fail), a message, and an optional detail line (e.g. the specific value or error behind the check).
5. If the check run itself fails (not the same as a failing check — this is a request error), an error banner appears with a retry button.
6. Use the refresh control in the header to re-fetch the VM list if it's out of date.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Webhooks

Purpose

Webhook Configuration — manage outbound webhooks that notify an external endpoint (generic HTTP, Slack, or Discord) when specific VM and backup events occur.

When to use it

- To wire VM lifecycle events (started, stopped, created, deleted) or backup results (completed, failed) into Slack, Discord, or your own HTTP endpoint
- To verify a webhook endpoint is reachable and correctly configured before relying on it
- To audit which webhooks exist, what events they listen for, and whether they're enabled

How to get there

- Route / id: `/webhooks`
- Nav: **Tools** → **Webhooks** (sidebar, command palette, or desktop nav)

Operate from the console (UX)

1. **Add Webhook** opens a form: destination URL, delivery type (generic / Slack / Discord), and a multi-select of events (`vm.started` , `vm.stopped` , `vm.created` , `vm.deleted` , `backup.completed` , `backup.failed`). A URL and at least one event are required — validation errors show inline before it will save.
2. Saving posts the new webhook (`POST /api/webhooks` , created enabled by default) and refreshes the list.
3. Each configured webhook shows its URL (with a copy button), a type badge, its subscribed event tags, and an enabled/disabled indicator.
4. **Test** sends a one-off test delivery (`POST /api/webhooks/test`) and reports delivered/failed inline on that webhook.
5. The trash icon asks for confirmation, then deletes the webhook (`DELETE /api/webhooks/{id}`).
6. Use the refresh control in the header to re-fetch the webhook list.

If the page stays empty, check service health, auth configuration, and that dependencies for this domain are installed.

Related pages

- [Getting Started](#)
 - [Page index](#)
-

Zyvor Fabric — Complete page index

Every primary navigable dashboard route.

Generated: 2026-09-01 · 84 routes

Regenerate: `node scripts/customer-docs/generate-page-index.mjs`

Core

Page	Route	Purpose	Guide
Dashboard	<code>/</code>	Dashboard — Core surface.	Open
Favorites	<code>/favorites</code>	Favorites — Core surface.	Open
Virtual Machines	<code>/vms</code>	Virtual Machines — Core surface.	Open
Machines	<code>/machines</code>	Machines — Core surface.	Open
Profiles	<code>/profiles</code>	Profiles — Core surface.	Open
Datacenters	<code>/datacenters</code>	Datacenters — Core surface.	Open
VM Browser	<code>/vm-browser</code>	VM Browser — Core surface.	Open
Create VM	<code>/create</code>	Create VM — Core surface.	Open
VM Console	<code>/vms/:name/console</code>	VM Console — Core surface.	—
Settings	<code>/settings</code>	Settings — Core product and console preferences.	Open
VM Wizard	<code>/vm-wizard</code>	VM Wizard — guided VM creation flow.	Open

Infrastructure

Page	Route	Purpose	Guide
Network	<code>/network</code>	Network — Infrastructure surface.	Open
Net Security	<code>/network-security</code>	Net Security — Infrastructure surface.	Open
Storage	<code>/storage</code>	Storage — Infrastructure surface.	Open
Storage Pools	<code>/storage-pools</code>	Storage Pools — Infrastructure surface.	Open
Distributed Storage	<code>/distributed-storage</code>	Distributed Storage — Infrastructure surface.	Open
Resource Pools	<code>/resource-pools</code>	Resource Pools — Infrastructure surface.	Open
System	<code>/system</code>	System — Infrastructure surface.	Open
System Health	<code>/system-health</code>	System Health — Infrastructure surface.	Open
Containers	<code>/containers</code>	Containers — Infrastructure surface.	Open
Zones	<code>/zones</code>	Zones — Infrastructure surface for placement domains.	Open
VM Pools	<code>/vm-pools</code>	VM Pools — warm / capacity pools for fast provisioning.	Open

Operations

Page	Route	Purpose	Guide
DRS	/drs	DRS — Operations surface.	Open
Fault Tolerance	/fault-tolerance	Fault Tolerance — Operations surface.	Open
Replication	/replication	Replication — Operations surface.	Open
Site Recovery	/site-recovery	Site Recovery — Operations surface.	Open
Migrations	/migrations	Migrations — Operations surface.	Open
Migration Wizard	/migration-wizard	Migration Wizard — Operations surface.	Open
Templates	/templates	Templates — Operations surface.	Open
Content Library	/content-library	Content Library — Operations surface.	Open
Schedules	/schedules	Schedules — Operations surface.	Open
Autoscale	/autoscale	Autoscale — Operations surface.	Open
Snapshots	/snapshots	Snapshots — Operations surface.	Open
Backups	/backups	Backups — Operations surface.	Open
Quotas	/quotas	Quotas — Operations surface.	Open
Lifecycle	/lifecycle	Lifecycle — Operations surface.	Open
Bulk Operations	/bulk-operations	Bulk Operations — Operations surface.	Open

Monitoring

Page	Route	Purpose	Guide
Logs	/logs	Logs — Monitoring surface.	Open
Analytics	/analytics	Analytics — Monitoring surface.	Open
Audit	/audit	Audit — Monitoring surface.	Open
Notifications	/notifications	Notifications — Monitoring surface.	Open
Alerts	/alerts	Alerts — Monitoring surface.	Open
Timeline	/timeline	Timeline — Monitoring surface.	Open
Processes	/processes	Processes — Monitoring surface.	Open
Kernel	/kernel	Kernel — Monitoring surface.	Open
Debug Tools	/debug	Debug Tools — Monitoring surface.	Open
Explain	/explain	Explain — Monitoring surface.	Open
Live Metrics	/live-metrics	Live Metrics — Monitoring surface.	Open
Event Stream	/event-stream	Event Stream — Monitoring surface.	Open
Optimizer	/resource-optimizer	Optimizer — Monitoring surface.	Open
Capacity	/capacity-planning	Capacity — Monitoring surface.	Open

Page	Route	Purpose	Guide
Service Map	/service-map	Service Map — Monitoring surface.	Open

Security

Page	Route	Purpose	Guide
Security Dashboard	/security-dashboard	Security Dashboard — Security surface.	Open
Encryption	/encryption	Encryption — Security surface.	Open
Certificates	/certificates	Certificates — Security surface.	Open
Compliance	/compliance	Compliance — Security surface.	Open
Access Control	/access-control	Access Control — Security surface.	Open
Plugins	/plugins	Plugins — Security surface.	Open

Tools

Page	Route	Purpose	Guide
Webhooks	/webhooks	Webhooks — Tools surface.	Open
Cost Estimator	/cost-estimator	Cost Estimator — Tools surface.	Open
VM Compare	/vm-compare	VM Compare — Tools surface.	Open
VM Health Check	/vm-healthcheck	VM Health Check — Tools surface.	Open
Notification Center	/notification-center	Notification Center — Tools surface.	Open
API Playground	/playground	API Playground — try Fabric APIs interactively.	Open

More — images, migrations & managers

Page	Route	Purpose	Guide
Readiness	/migration-readiness	Readiness — More — images, migrations & managers surface.	Open
History	/migration-history	History — More — images, migrations & managers surface.	Open
Report	/migration-report	Report — More — images, migrations & managers surface.	Open
Migration Templates	/migration-templates	Migration Templates — More — images, migrations & managers surface.	Open
Batch Migration	/batch-migration	Batch Migration — More — images, migrations & managers surface.	Open
ISO Images	/iso-images	ISO Images — More — images, migrations & managers surface.	Open
Upload Disk	/upload-disk	Upload Disk — More — images, migrations & managers surface.	Open
Download Disk	/download-disk	Download Disk — More — images, migrations & managers surface.	Open

Page	Route	Purpose	Guide
Image Builder	/image-builder	Image Builder — More — images, migrations & managers surface.	Open
Pipeline	/pipeline	Pipeline — More — images, migrations & managers surface.	Open
Disk Images	/disk-images	Disk Images — More — images, migrations & managers surface.	Open
Disk Converter	/disk-converter	Disk Converter — More — images, migrations & managers surface.	Open
Backup Scheduler	/backup-scheduler	Backup Scheduler — More — images, migrations & managers surface.	Open
Batch Import	/batch-import	Batch Import — More — images, migrations & managers surface.	Open
Snapshot Mgr	/snapshot-manager	Snapshot Mgr — More — images, migrations & managers surface.	Open
Storage Mgr	/storage-manager	Storage Mgr — More — images, migrations & managers surface.	Open
Manifest Builder	/manifest-builder	Manifest Builder — More — images, migrations & managers surface.	Open
Job Monitor	/job-monitor	Job Monitor — More — images, migrations & managers surface.	Open
Network Topology	/network-topology	Network Topology — More — images, migrations & managers surface.	Open

Auth

Page	Route	Purpose	Guide
Login	/login	Login — Auth surface.	Open

Related

- [Customer docs home](#)
- [Page-by-page guides](#)